Python

Ingyu Bahng

May 2026

Contents

1 Introduction to Python

Core Concept 1.1: Python 3.0

Following several major releases over preceding decades (*Python 0.9.0 in February 1991, Python 1.0 in January 1994, Python 2.0 in October 2000*), Python 3.0 was released in December 2008 and has since evolved through incremental updates to Python 3.14.5 as of May 10, 2026.

The largest and most impactful version change came with the **transition from Python 2 to Python 3**, requiring over a decade for a complete migration due to **deliberate backward incompatibility**. This breaking change made many developers hesitant to migrate given the substantial time and cost involved in a complete update to Python 3 syntax and logic in existing codebases.

Today, Python 3 represents the definitive global industry standard for development, while Python 2 is structurally obsolete. Python 2 officially reached its End-of-Life (EOL) on January 1, 2020, and no longer receives security patches, bug fixes, or official maintenance.

Core Concept 1.2: CPython vs Jython vs PyPy

The original Python interpreter was implemented in **C**, a lower-level language positioned above **assembly** and **machine code**. This C-based implementation, known as **CPython**, remains the standard reference version distributed via python.org.

Alternative implementations include **Jython**, built on Java for seamless Java ecosystem integration, and **PyPy**, which offers significantly faster execution for pure Python code. However, Jython remains limited to Python 2.7 compatibility, while PyPy exhibits reduced compatibility with C-based libraries (*Core Concept 6.1*), many of which are essential to major projects.

2 Data Types & Structures

2.1 Singular Data Types

Core Concept 2.1: Numerics

Numeric data types encompass the set of data representations used to encode quantitative values within Python. Python provides several built-in numeric forms, each made for different purposes.

```
>>> a,b,c,d = 10, 0b1010, 0o12, 0x0a
>>> print(a, b, c, d)
10 10 10 10
>>> print(type(a), type(b), type(c), type(d))
<class 'int'> <class 'int'> <class 'int'> <class 'int'>

>>> print(type(3.14))
<class 'float'>

>>> print(type(2 + 5j))
<class 'complex'>
```

Code 1: Numeric data types `int`, `float`, and `complex`

(a) `int`: Integers represent whole numbers without a fractional component. They may be positive, negative, or zero. For example, the values `7`, `-42`, and `0` are all classified as integers in Python.

Integers can also be represented in **binary** (*base-2*), **octal** (*base-8*), or **hexadecimal** (*base-16*) notation using the prefixes `0b`, `0o`, and `0x`, respectively. For instance, `0b1010`, `0o12`, and `0x0a` all correspond to the decimal integer value `10`.

(b) `float`: Floats represent real numbers containing decimal components. For instance, `3.14` and `-0.001` are floating-point values commonly used to represent continuous numerical quantities.

(c) `complex`: Complex numbers represent values composed of both real and imaginary components in the form `a + bi`, where `i` denotes the imaginary unit satisfying $i^2 = -1$. As an example, the expression `2 + 5j` represents a complex number, where `j` denotes the imaginary component.

Core Concept 2.2: Boolean

`bool` represents logical truth values consisting exclusively of `True` or `False`. They are primarily used in conditional expressions, logical operations, and program control flow. For example, the expression `5 > 3` evaluates to `True`, whereas `5 < 3` evaluates to `False`.

```
>>> print(type(True), type(False))
<class 'bool'> <class 'bool'>
```

Code 2: `bool` data type

Core Concept 2.3: None

`NoneType` data type represents the **absence** of a value or the **intentional lack** of an assigned object. Python provides the singleton value `None` to denote null or undefined states within a program.

```
>>> print(type(None))
<class 'NoneType'>
```

Code 3: `None` data type

2.1.1 Strings

Core Concept 2.4: String

The `str` data type is an ordered sequence of textual characters surrounded by quotation marks, serving as the fundamental data type for text. For example, `"hello world"` constitutes a string literal. String objects support a variety of methods and operations for transformation and analysis.

```
>>> print(type("hello world"))
<class 'str'>
```

Code 4: `str` data types

(a) **Indexing and Slicing**: Accesses individual characters at specified positions using zero-based numerical indices, or extracts contiguous subsequences by specifying start and end positions.

```
>>> example_string = "hello world"
>>> print(example_string[0])
h
>>> print(example_string[3:9])
lo wor
```

Code 5: String indexing and slicing

(b) **Repetition**: Creates a new string by concatenating multiple copies of the original string.

```
>>> example_string = "hello world"
>>> print(example_string*2)
hello worldhello world
```

Code 6: String repetition

(c) **Length**: `len()` returns the total number of characters contained within the string.

```
>>> example_string = "hello world"
>>> print(len(example_string))
11
```

Code 7: Retrieving string length through `len()`

(d) **Strip**: `.strip()` removes leading and trailing characters from the string. By default, `.strip()` removes whitespace.

```
>>> example_string = "   hello world   "
>>> print(example_string)
   hello world
>>> stripped_string = example_string.strip()
>>> print(stripped_string)
hello world
```

Code 8: Removing leading and trailing whitespace from a string with `.strip()`

(e) **Find**: `.find()` locates the first occurrence of a specified substring and returns its starting index position.

```
>>> example_string = "hello world"
>>> substring = "l"
>>> example_string.find(substring)
2
```

Code 9: Finding a substring within a string through `.find()`

(f) **Count**: `.count()` returns the number of non-overlapping occurrences of a specified substring within the string.

```
>>> example_string = "hello world"
>>> substring = "l"
>>> example_string.count(substring)
3
```

Code 10: Determining a substring count within a string using `.count()`

(g) **Upper/Lower/Title**: `.upper()`, `.lower()`, and `.title()` converts all characters to uppercase, lowercase, or title case (*capitalizing the first letter of each word*), respectively.

```
>>> example_string = "hello world"
>>> print(example_string.upper())
HELLO WORLD
>>> print(example_string.lower())
hello world
>>> print(example_string.title())
Hello World
```

Code 11: Converting a string to upper, lower, and title case using `.upper()`, `.lower()`, and `.title()`

(h) **Split**: `.split()` divides the string into a list of substrings based on a specified delimiter character or whitespace.

```
>>> example_string = "hello world this is an example string"
>>> print(example_string.split())
['hello', 'world', 'this', 'is', 'an', 'example', 'string']
```

Code 12: Spliting a string into components using `.split()`

Core Concept 2.5: Escape Characters

Certain special string character sequences begin with a backslash and represent non-printable or reserved characters. For instance, `\n` for newline, `\t` for tab, `\\` for the \character, and `\"` for the " character.

```
>>> example_string = "hello world\n\nhello world\\ \t \""
>>> print(example_string)
hello world

hello world\          "
```

Code 13: Special escape characters

Core Concept 2.6: String Types

Python provides several string literal prefixes that modify how the interpreter parses and stores the resulting object. The three primary prefixes are `f`, `r`, and `b`, which may also be combined in certain valid pairings.

(a) `f`: **f-strings** allow arbitrary Python expressions to be embedded directly within a string literal by enclosing them in curly braces `{}`. Expressions are evaluated at runtime and their results interpolated into the surrounding string.

```
>>> name = "Ada"
>>> print(f"Hello, {name}!")
Hello, Ada!
>>> print(f"Pi approx {3.14159:.2f}")
Pi approx 3.14
>>> print(f"{name=}")
name='Ada'
```

Code 14: f-string demonstration

(b) `r`: **r-strings** instruct Python to treat every backslash (*Core Concept 2.5*) as a literal character rather than

the start of an escape sequence.

```
>>> print("line1\nline2")
line1
line2
>>> print(r"line1\nline2")
line1\nline2
```

Code 15: r-string demonstration

(c) `b`: **b-strings** produce a `bytes` object (*Core Concept 2.13*) rather than a `str`. Each element must be an ASCII character or an escape sequence. This type is used extensively in binary file I/O, network communication, and low-level data processing.

```
>>> b = b"hello"
>>> print(type(b))
<class 'bytes'>
>>> print(b[0])
104
```

Code 16: b-string demonstration

Prefixes can also be combined in order to use the functionality of multiple string types concurrently. The ordering of these prefixes is insignificant, meaning that the `rf` prefix is equivalent to the `fr` prefix.

Some commonly used combined string prefixes are `rf`—which is typically present in dynamic regex (*Core Concept 10.1*) patterns—and `rb`, which is used when the corresponding binary string includes literal backslashes.

```
>>> user_id = "dev_88"
>>> dynamic_path = rf"C:\Users\admin\desktop\{user_id}"
>>> print(f"Dynamic Path: {dynamic_path}")
Dynamic Path: C:\Users\admin\desktop\dev_88

>>> binary_regex_pattern = rb"FindThis\x00\x01"
>>> print(f"Raw Byte Pattern: {binary_regex_pattern}")
Raw Byte Pattern: b'FindThis\\x00\\x01'
```

Code 17: String prefix combination `rf` and `rb`

Core Concept 2.7: String Formatting

2.2 Sequence Data Types

Core Concept 2.8: Lists

A `list` is an ordered, **mutable** (*modifiable after creation*) sequence that can contain elements of any data type. Lists are defined using square brackets and support dynamic modification of their contents. For example, `[1, 2, 3, "hello"]` constitutes a list literal containing both integers and a string.

(a) **Indexing and Slicing**: Accesses individual elements or subsequences using zero-based indices, analogous to string operations.

```
>>> example_list = [1, 2, 3, 4, 5]
>>> print(example_list[0])
1
>>> print(example_list[1:4])
[2, 3, 4]
```

Code 18: List indexing and slicing

(b) **Element Addition & Removal**: `.append()` and `.insert()` both add a single element to the end of the list, modifying the list in place, but `.insert()` allows the addition to occur at a specified index position, shifting subsequent elements rightward. Conversely, `.remove()` deletes the first occurrence of a specified value from the list.

```
>>> example_list = [1, 2, 3]
>>> example_list.append(4) # append
>>> print(example_list)
[1, 2, 3, 4]
>>> example_list.insert(2, 5) # inserting 3 at index 2
>>> print(example_list)
[1, 2, 5, 3, 4]
>>> example_list.remove(2) # removal
>>> print(example_list)
[1, 5, 3, 4]
```

Code 19: List element addition through `.append()` / `.insert()` and removal through `.remove()`

(c) **Sort**: `.sort()` Arranges list elements in a specified order, modifying the list in place. By default, `.sort()` arranges the list in ascending order.

```
>>> example_list = [3, 1, 4, 1, 5, 9]
>>> example_list.sort() # ascending order
>>> print(example_list)
[1, 1, 3, 4, 5, 9]
```

Code 20: List sorting using `.sort()`

(d) **Join**: `.join()` concatenates list elements into a single **string** using a specified delimiter.

```
>>> example_list = ["hello", "world", "example"]
>>> print(" ".join(example_list))
hello world example
```

Code 21: Joining a list of strings using `.join()`

(e) **List Comprehension**: Constructs new lists using concise inline syntax that applies an expression to each element of an iterable (*Core Concept 3.2*).

```
>>> numbers = [1, 2, 3, 4, 5]
>>> print([x*2 for x in numbers])
[2, 4, 6, 8, 10]
```

Code 22: List comprehension

Core Concept 2.9: Tuples

A `tuple` is an ordered, **immutable** (*cannot be modified after creation*) sequence that can contain elements of any data type, making it suitable for fixed data structures that should not be modified. For example, `(1, 2, 3, "hello")` constitutes a tuple literal.

(a) **Indexing and Slicing**: Accesses individual elements or subsequences using zero-based indices, identical to list operations.

```
>>> example_tuple = (1, 2, 3, 4, 5)
>>> print(example_tuple[0], example_tuple[1:4])
1 (2, 3, 4)
```

Code 23: Tuple indexing and slicing

(b) **Tuple Packing and Unpacking**: Assigns multiple values simultaneously or extracts tuple elements into separate variables.

```
>>> x,y = (10, 20)
>>> print(x)
10
```

Code 24: Tuple packing and unpacking

Core Concept 2.10: Sets

A `set` is an **unordered** collection of **unique** elements that support mathematical set operations. Defined using curly braces or the `set()` constructor, sets automatically **eliminate duplicate values**. For example, `{1, 2, 3}` constitutes a set literal.

(a) **Element Addition & Removal**: `.add()` and `.update()` inserts a single element or multiple elements, respectively into the set. Conversely, `.remove()` deletes a specified element, raising an error if the element does not exist.

```
>>> example_set = {1, 2, 3}

>>> example_set.add(4) # singular addition
>>> print(example_set)
{1, 2, 3, 4}

>>> example_set.update([3, 4, 5]) # multiple addition
>>> print(example_set)
{1, 2, 3, 4, 5}

>>> example_set.remove(3) # removal
>>> print(example_set)
{1, 2, 4, 5}
```

Code 25: Set element addition through `.add()` / `.update()` and removal through `.remove()`

(b) **Set Operations**: Performs mathematical set operations including union, intersection, and difference.

```
>>> set_a = {1, 2, 3}
>>> set_b = {3, 4, 5}
>>> print(set_a.union(set_b))
```

```
{1, 2, 3, 4, 5}
>>> print(set_a.intersection(set_b))
{3}
```

Code 26: Set operations; `.union()` and `.intersection()`

(c) **Frozenset**: `frozenset()` creates an **immutable** version of a set that cannot be modified after creation.

```
>>> fset = frozenset({1,2,3})
>>> print(type(fset))
<class 'frozenset'>
```

Code 27: Frozenset using `frozenset()`

Core Concept 2.11: Dictionaries

A `dict` is an unordered collection of **key-value pairs** that enable efficient data lookup by key. Defined using curly braces with colon-separated pairs, dictionaries map unique keys to associated values. For example, `{"name": "Alice", "age": 30}` constitutes a dictionary literal.

(a) **Accessing Values**: Retrieves the value associated with a specified key using bracket notation.

```
>>> example_dict = {"name": "Alice", "age": 30}
>>> print(example_dict["name"])
Alice
```

Code 28: Accessing dictionary values

(b) **Adding and Modifying**: Assigns new key-value pairs or updates existing values.

```
>>> example_dict = {"name": "Alice"}
>>> example_dict["age"] = 30
>>> print(example_dict)
{'name': 'Alice', 'age': 30}
```

Code 29: Dictionary item addition and modification

(c) **Retrieving Dictionary Elements**: `.keys()`, `.values()`, and `.items()` returns a view object containing all dictionary keys, values, and key-value pairs, respectively.

```
>>> example_dict = {"name": "Alice", "age": 30}
>>> print(example_dict.keys(), example_dict.values())
dict_keys(['name', 'age']) dict_values(['Alice', 30])
>>> print(example_dict.items())
dict_items([('name', 'Alice'), ('age', 30)])
```

Code 30: Dictionary element retrieval using `.keys()`, `.values()`, and `.items()`

Core Concept 2.12: Ranges

A `range` is an memory-efficient immutable sequence of integers commonly used for loop iteration. It is

generated using the `range()` function with `start`, `stop`, and optional `step` parameters.

The **basic range** creates a sequence of integers from 0 up to (*but not including*) a specified endpoint.

```
>>> print(list(range(5)))
[0, 1, 2, 3, 4]
```

Code 31: Basic range creation

However, you can also begin the range from a specified `start` value. Additionally, the `step` parameter allows for a specified increment between consecutive values.

```
>>> example_range = range(0, 10, 2)
>>> print(list(range(0, 10, 2)))
[0, 2, 4, 6, 8]
```

Code 32: Range with `start` and `step`

Core Concept 2.13: Bytes & Bytearrays

`bytes` and `bytearray` objects are sequences of integer values representing raw binary data. `bytes` objects are immutable, while `bytearrays` support modification. These types are essential for binary file operations, network protocols, and low-level data manipulation.

(a) **Bytes**: `b""` creates an immutable sequence of bytes from a string using specified encoding.

```
>>> example_bytes = b"hello"
>>> print(example_bytes, type(example_bytes))
b'hello' <class 'bytes'>
```

Code 33: Bytes using `b""` syntax

(b) **Bytearray**: `bytearray()` creates a mutable sequence of bytes that can be modified after creation.

```
>>> example_bytearray = bytearray(b"hello")
>>> example_bytearray[0] = 72 # the byte value for "H" is 72 in ASCII/UTF-8
>>> print(example_bytearray)
bytearray(b'Hello')
```

Code 34: Bytearray creation through `bytearray()`

(c) **Encoding and Decoding**: Converts between strings and bytes using character encodings.

```
>>> text = "hello"
>>> encoded = text.encode('utf-8')
>>> print(encoded)
b'hello'
>>> print(encoded.decode('utf-8'))
hello
```

Code 35: Encoding and decoding strings to bytes through `.encode()` and `.decode()`

2.3 Operators & Operands

Core Concept 2.14: Arithmetic Operators

Arithmetic operators perform mathematical calculations on numeric operands, returning numeric results.

(a) **Basic Arithmetic**: `+`, `-`, `*`, `/` perform addition, subtraction, multiplication, and division on numeric operands, respectively. Division always returns a floating-point result. Note that `+` and `*` also operate on strings for concatenation and repetition (*Core Concept 2.4*).

```
>>> x,y = 10, 3
>>> print(x + y, x - y)
13 7
>>> print(x * y, x / y)
30 3.3333333333333335
```

Code 36: Basic arithmetic operators

(b) **Modulus**: `%` returns the remainder after division of the first operand by the second.

```
>>> x,y = 17, 5
>>> print(x % y)
2
```

Code 37: Modulus operator

(c) **Power**: `**` raises the first operand to the power of the second operand.

```
>>> x,y = 2, 3
>>> print(x ** y)
8
```

Code 38: Power operator

(d) **Floor Division**: `//` divides the first operand by the second and rounds down to the nearest integer.

```
>>> x,y = 17, 5
>>> print(x // y)
3
```

Code 39: Floor division operator

Core Concept 2.15: Assignment Operators

Assignment operators assign values to variables and can combine assignment with arithmetic operations for concise code.

(a) **Basic Assignment**: `=` assigns the value on the right to the variable on the left.

```
>>> x = 10
```

Code 40: Basic assignment operator

*By convention, identifiers intended to be treated as **constants**—such as API keys or authentication credentials—are*

written in full capitals within the Python community, serving as an explicit signal to other developers that the value is not to be mutated at any point during program execution.

(b) **Compound Assignment Operators**: `+=`, `-=`, `*=`, `/=`, `%=`, `//=`, and `**=` combine each arithmetic operator (*Core Concept 2.14*) with assignment, performing the operation on the left operand with the right operand and assigning the result back to the left operand.

```
>>> x = 10
>>> x += 3 # addition
>>> print(x)
13
>>> x -= 2 # subtraction
>>> print(x)
11
>>> x *= 2 # multiplication
>>> print(x)
22
>>> x /= 4 # division
>>> print(x)
5.5
>>> # the same logic follow for modulus, floor division, and power
```

Code 41: Compound assignment operators

Core Concept 2.16: Comparison Operators

Comparison operators compare two operands and return a Boolean value (*Core Concept 2.2*) indicating the result of the comparison.

(a) **Equal To**: `==` returns `True` if both operands are equal in value.

```
>>> x,y = 5, 5
>>> print(x == y)
True
>>> print(x == 3)
False
```

Code 42: Equality comparison operator

(b) **Not Equal To**: `!=` returns `True` if operands are not equal in value.

```
>>> x,y = 5,3
>>> print(x != y)
True
>>> print(x != 5)
False
```

Code 43: Inequality comparison operator

(c) **Relational Comparisons**: `<`, `>`, `<=`, and `>=` compare the magnitude of two operands, returning `True` if the left operand is less than, greater than, less than or equal to, or greater than or equal to the right operand, respectively.

```
>>> x,y = 5,3
>>> print(x < y)
False
>>> print(x > y)
```

```
True
>>> print(x <= 5)
True
>>> print(x >= 5)
True
```

Code 44: Relational comparison operators

Core Concept 2.17: Logical Operators

Logical operators combine or modify Boolean expressions, returning Boolean values based on logical relationships.

(a) **And**: `and` returns `True` only if both operands are `True`.

```
>>> x,y = True, False
>>> print(x and x)
True
>>> print(x and y)
False
```

Code 45: `add` logical operator

(b) **Or**: `or` returns `True` if at least one operand is `True`.

```
>>> x,y = True, False
>>> print(x or y)
True
>>> print(y or y)
False
```

Code 46: `or` logical operator

(c) **Not**: `not` returns the opposite Boolean value of the operand.

```
>>> x,y = True, False
>>> print(not x, not y)
False True
```

Code 47: `not` logical operator

3 Control Flow

3.1 Conditionals

Core Concept 3.1: if, elif, else

Conditional statements enable selective code execution based on Boolean conditions. An `if` statement evaluates a specified condition and executes its code block only when the condition evaluates to `True`. The `elif` (*else-if*) clause allows sequential evaluation of multiple conditions, executing only when all preceding conditions evaluate to `False`. The `else` clause provides a default code block that executes when all preceding

conditions fail.

```
>>> x, y = 10, 5
>>> if x > y:
...     print("x is greater than y") # conditional ends here
... elif y > x:
...     print("y is greater than x")
... else:
...     print("x is equal to y")
...
x is greater than y
```

Code 48: Conditional `if` / `elif` / `else` statement block

3.2 Loops

Core Concept 3.2: for, break, continue

The `for` loop enables iteration over elements in an iterable sequence (*lists, tuples, strings, ranges*). Each iteration assigns the current element to a loop variable and executes the indented code block. The loop terminates automatically upon processing all elements, or manually via the `break` statement, which immediately exits the loop regardless of whether all elements have been processed.

```
>>> for num in [1, 2, 3, 4]:
...     if num == 3:
...         break
...     print(num)
...
1
2
```

Code 49: `for` loop with `break`

Conversely, the `continue` statement skips the remainder of the current iteration and proceeds immediately to the next iteration without terminating the loop entirely.

```
>>> for num in [1, 2, 3]:
...     if num == 2:
...         continue
...     print(num)
...
1
3
```

Code 50: `for` loop with `continue`

Core Concept 3.3: while

The `while` loop repeatedly executes its code block as long as a specified Boolean condition remains `True`. Unlike `for` loops that iterate over finite sequences, `while` loops continue indefinitely until their condition evaluates to `False`, making them suitable for scenarios where the iteration count is not predetermined.

```
>>> x = 0
>>> while x < 3:
...     print(x)
...     x += 1
...
0
1
2
```

Code 51: Basic `while` loop

The `break` and `continue` statements (*Core Concept 3.2*) function identically within `while` loops, allowing for premature loop termination or iteration skipping, respectively.

```
>>> x = 0
>>> while x < 5:
...     x += 1
...     if x == 2:
...         continue
...     if x == 4:
...         break
...     print(x)
...
1
3
```

Code 52: `while` loop with `continue` and `break`

4 Functions

Core Concept 4.1: def

The `def` keyword defines a **function**—a reusable block of code that executes when called. Functions accept input through **parameters** (*variables defined in the function*), process data according to their internal logic, and optionally return output values using the `return` statement.

```
>>> def greet(name):
...     return f"Hello, {name}!"
...
>>> print(greet("Alice"))
Hello, Alice!
>>> print(greet("Bob"))
Hello, Bob!
```

Code 53: Basic function definition through `def`

Python supports **positional parameters** (*matched by position*) and **keyword parameters** (*matched by name*). Additionally, default parameter values allow functions to be called with fewer arguments than defined parameters, providing fallback values when arguments are omitted.

```
>>> def add(x, y):
...     return x + y
...
>>> print(add(3, 5))
8
```

```
>>> print(add(x=10, y=20))
30

>>> def greet(name, greeting="Hello"):
...     return f"{greeting}, {name}!"
...
>>> print(greet("Alice"))
Hello, Alice!
>>> print(greet("Bob", "Hi"))
Hi, Bob!
```

Code 54: `def` function positional and keyword arguments

`*args` and `**kwargs` enable functions to accept variable numbers of arguments. `*args` collects excess positional arguments into a tuple, while `**kwargs` collects excess keyword arguments into a dictionary.

```
>>> def print_args(*args):
...     for arg in args:
...         print(arg)
...
>>> print_args(1, 2, 3, 4, 5)
1
2
3
4
5

>>> def print_kwargs(**kwargs):
...     for key, value in kwargs.items():
...         print(f"{key}: {value}")
...
>>> print_kwargs(name="Alice", age=30, city="NYC")
name: Alice
age: 30
city: NYC
```

Code 55: `*args` ; `def` function variable position arguments

Functions can utilize both `*args` and `**kwargs` simultaneously to achieve maximum flexibility in parameter handling.

```
>>> def flexible_func(x, y, *args, **kwargs):
...     print(f"x: {x}, y: {y}")
...     print(f"args: {args}")
...     print(f"kwargs: {kwargs}")
...
>>> flexible_func(1, 2, 3, 4, 5, name="Alice", age=30)
x: 1, y: 2
args: (3, 4, 5)
kwargs: {'name': 'Alice', 'age': 30}
```

Code 56: `**kwargs` ; `def` function variable keyword arguments

Core Concept 4.2: lambda

The `lambda` keyword creates **anonymous functions**—small, unnamed functions defined in a single expression. Lambda functions are syntactically restricted to a single expression and implicitly return the result of that expression, making them suitable for simple operations that don't warrant full function definitions (*Core Concept 4.1*).

```
>>> def add(x, y): # regular function
...     return x + y
...
>>> add_lambda = lambda x, y: x + y # equivalent lambda function
>>> print(add(3, 5))
8
>>> print(add_lambda(3, 5))
8
```

Code 57: `lambda` function

Lambda functions are commonly used as **arguments to higher-order functions** that accept functions as parameters, such as `map()`, `filter()`, and `sorted()`.

```
>>> numbers = [1, 2, 3, 4, 5]

>>> squared = list(map(lambda x: x**2, numbers)) # map: apply function to each element
>>> print(squared)
[1, 4, 9, 16, 25]

>>> evens = list(filter(lambda x: x % 2 == 0, numbers)) # filter: keep elements where
 ↪ function returns True
>>> print(evens)
[2, 4]

>>> words = ["apple", "pie", "zoo", "a"] # sorted: custom sorting key
>>> sorted_by_length = sorted(words, key=lambda x: len(x))
>>> print(sorted_by_length)
['a', 'pie', 'zoo', 'apple']
```

Code 58: `lambda` with `map()` function

Lambda functions can **reference variables from their enclosing scope**, meaning that they have the ability to access and utilize variables that were defined outside of its own internal parameter list and thus creating closures that capture external states.

```
>>> def make_multiplier(n):
...     return lambda x: x * n
...
>>> times_two = make_multiplier(2)
>>> print(times_two(5))
10
```

Code 59: `lambda` function referencing variables from their enclosing scope

Core Concept 4.3: Decorators

Decorators are functions that **modify** or enhance the behavior of other functions without altering their source code. Decorators wrap the target function, adding functionality before or after its execution, and are applied using the `@` syntax.

```
>>> def uppercase_decorator(func):
...     def wrapper(*args, **kwargs):
...         result = func(*args, **kwargs)
...         return result.upper()
...     return wrapper
...
>>> @uppercase_decorator
... def greet(name):
...     return f"hello, {name}"
...
>>> print(greet("alice"))
HELLO, ALICE
```

Code 60: Basic uppercase decorator

Decorators can accept arguments by adding an additional layer of function nesting, enabling parameterized decoration behavior.

```
>>> def repeat(times):
...     def decorator(func):
...         def wrapper(*args, **kwargs):
...             for _ in range(times):
...                 result = func(*args, **kwargs)
...             return result
...         return wrapper
...     return decorator
...
>>> @repeat(times=2)
... def say_hello():
...     print("Hello!")
...
>>> say_hello()
Hello!
Hello!
```

Code 61: Parameterized multiplier decorator

Multiple decorators can be stacked on a single function, applying transformations in bottom-to-top order (*the decorator closest to the function executes first*).

```
>>> def bold(func):
...     def wrapper(*args, **kwargs):
...         return f"**{func(*args, **kwargs)}**"
...     return wrapper
...
>>> def italic(func):
...     def wrapper(*args, **kwargs):
...         return f"_{func(*args, **kwargs)}_"
...     return wrapper
...
>>> @bold
... @italic
... def text():
...     return "Hello"
...
```

```
>>> print(text())
**_Hello_**
```

Code 62: Stacked decorators

Core Concept 4.4: Generators

Generators are functions that use the `yield` keyword to produce a sequence of values lazily, generating each value **on-demand** rather than computing and storing all values in memory simultaneously. This enables efficient iteration over large or infinite sequences. Unlike regular functions that return once and terminate, generators **maintain their state** between `yield` statements, resuming execution from where they left off when the next value is requested.

```
>>> def count_up_to(n):
...     count = 1
...     while count <= n:
...         yield count
...         count += 1
...
>>> count_gen = count_up_to(10)

>>> for _ in range(2):
...     print(next(count_gen))
...
1
2
>>> for _ in range(3): # starts from where it left off in the previous for loop
...     print(next(count_gen))
...
3
4
5
```

Code 63: Generator state maintenance using `next()`

Generator expressions provide a concise syntax for creating generators, analogous to list comprehensions but with parentheses instead of brackets, **producing values lazily without constructing the entire sequence in memory**. Once a generator yields a value, that value is immediately forgotten by the generator. Once you have looped through the entire generator, it is completely empty. In programming, this is known as **generator exhaustion**.

```
>>> squares_gen = (x**2 for x in range(3))

>>> print(list(squares_gen))  # first time: we consume the generator
[0, 1, 4]

>>> print(list(squares_gen))  # second time: the generator is already exhausted
[]
```

Code 64: Generator exhaustion

5 Modules & Packages

Core Concept 5.1: Modules

A **module** is a single Python source file—a `.py` file—that encapsulates a logically related collection of definitions, including functions, classes, and variables, which may be imported and reused across other Python programs via the `import` statement.

To illustrate, consider a file named `math_utils.py` containing the following definitions.

```
>>> # math_utils.py
>>> def add(a, b):
...     return a + b
... def subtract(a, b):
  File "<stdin>", line 3
    def subtract(a, b):
    ^^^
SyntaxError: invalid syntax
>>>     return a - b
  File "<stdin>", line 1
    return a - b
IndentationError: unexpected indent
```

Code 65: `math_utils.py` contents

A separate file—`main.py`, for instance—residing in the same directory may then access the definitions within `math_utils.py` via the `import` statement. There are two conventional forms through which this may be achieved.

(a) `import math_utils` grants access to **all** definitions within `math_utils.py`; however, each object must be qualified with the `math_utils.` prefix upon invocation.

```
>>> # main.py
>>> import math_utils
>>> print(math_utils.add(3,2))
5
```

Code 66: `import _` syntax module importing

To import only specific definitions rather than the module in its entirety, the `import _._` syntax may be used in place of the bare `import _` form.

(b) `from math_utils import *` similarly exposes **all** definitions from `math_utils.py`, but imports them directly into the calling file's namespace, eliminating the need for a qualifying prefix on each invocation.

```
>>> # main.py
>>> from math_utils import *
>>> print(add(3,2))
5
```

Code 67: `from _ import _` syntax module importing

This form also supports selective importing, wherein only specific definitions are brought into the calling namespace by replacing the wildcard `*` with the desired object or function.

Both import forms additionally support the `as` keyword, which binds the imported module to a user-defined

alias and substitutes it in place of the fully qualified module name across all subsequent references. For instance, `import math_utils as mu` permits the use of the abbreviated prefix `mu.` in place of `math_utils.` throughout the calling file, reducing characters without sacrificing the namespace qualification that distinguishes the module's definitions from those of the local scope.

In summary, the `import _` form imports a module by filename—excluding the `.py` extension—making its contents accessible through dot-qualified notation, whereas the `from _ import _` form brings definitions directly into the calling namespace, foregoing the need for prefix qualification. The latter, however, **carries the risk of namespace collisions** when multiple modules define identically named objects, and is therefore considered **poor practice** in larger projects and codebases.

Core Concept 5.2: Packages

A **package** is a structured directory of **one or more related modules**, unified under a common namespace (*the package name*), allowing developers to organise modules **hierarchically**, making large codebases navigable and preventing namespace collisions between identically named modules residing in different directories. An extremely basic package directory structure looks like the following.

```
my_package/
├── __init__.py
├── math_utils.py
└── string_utils.py
```

Modules within a package are accessed using dot-separated **path** notation that mirrors the directory hierarchy, with the package name serving as the top-level namespace. This structure ensures that `math_utils` within `my_package` remains unambiguously distinct from any identically named module elsewhere in the project, such as `string_utils`.

6 Standard Library

Core Concept 6.1: Libraries

A **library** is a broadly distributed, reusable collection of packages and modules that provides generalised functionality intended to be shared across many independent projects. Unlike a module or package, a library is not a formally defined construct within the Python; rather, it is an organisational and distributional concept. The distinction between a package and a library is primarily one of scale and intent: a package is a unit of code organisation within a project, whereas a library is a distribution of functionality designed for broad external consumption.

Core Concept 6.2: Standard Library

The **Python Standard Library** is a comprehensive collection of modules that comes with the Python interpreter upon installation, requiring no external package manager or configuration to access. Each module within the standard library addresses a distinct domain of common programming tasks, spanning areas such as file I/O, networking, data serialisation, concurrency, mathematics, and string manipulation, among others.

The modules and packages introduced in the following section will recur throughout the subsequent material, appearing either as the primary subject of discussion or as supplementary tooling within broader coding demonstrations.

6.1 Data Serialization & Storage

6.1.1 json

Core Concept 6.3: json

The `json` module provides a standardised interface for serialising Python objects into JSON (JavaScript Object Notation) format and deserialising JSON-formatted strings back into native Python objects. It is most commonly used when exchanging data with web APIs or writing structured data to a file.

(a) `json.dumps(obj)` serializes a Python object into a JSON-formatted string. You can also serialize a Python object and write the result directly to a file object `fp` through `json.dump(obj, fp)`.

```
>>> import json

>>> data = {'name': 'Alice', 'age': 30}

>>> json_string = json.dumps(data) # Python object -> json string
>>> print(json_string)
{"name": "Alice", "age": 30}

>>> with open('data.json', 'w') as fp: # Python object -> json file
...     json.dump(data, fp)
...
```

Code 68: Serializing a Python object to JSON using `json.dumps()` and `json.dump()`

(b) `json.loads(obj)`, on the other hand, deserializes a JSON-formatted string back into a native Python object. Likewise, you can write the result directly to a file object `fp` thorugh `json.load(fp)`.

```
>>> import json

>>> json_string = '{"name": "Alice", "age": 30}'

>>> data_from_string = json.loads(json_string) # json string -> Python object
>>> print(data_from_string)
{'name': 'Alice', 'age': 30}

>>> with open('data.json', 'r') as fp: # json file -> Python object
...     data_from_file = json.load(fp)
...
>>> print(data_from_file)
{'name': 'Alice', 'age': 30}
```

Code 69: Deserializing a JSON-formatted string to a Python object using `json.loads()` and `json.load()`

6.1.2 pickle

Core Concept 6.4: pickle

The `pickle` module implements a Python-specific binary serialisation protocol, enabling arbitrary Python objects—including custom class instances, functions, and data structures not supported by JSON—to be converted into a machine code and subsequently reconstructed. This process is referred to as **pickling** (*serialisation*) and **unpickling** (*deserialisation*) respectively.

A more detailed treatment of the pickling protocol is provided in Core Concept 9.3.

6.1.3 xml

Core Concept 6.5: xml

The `xml` module provides tooling for parsing, traversing, and constructing XML (eXtensible Markup Language) documents. The most practical sub-module for general use is `xml.etree.ElementTree`—often imported under the alian `ET` for brevity—which represents an XML document as a navigable tree of `Element` objects and is the idiomatic interface for XML processing within the standard library.

(a) `ET.parse(file)` parses an XML file and returns an `ElementTree` object representing the full document.

```
>>> import xml.etree.ElementTree as ET

>>> tree = ET.parse('inventory.xml')
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File "/opt/anaconda3/lib/python3.11/xml/etree/ElementTree.py", line 1219, in parse
    tree.parse(source, parser)
  File "/opt/anaconda3/lib/python3.11/xml/etree/ElementTree.py", line 570, in parse
    source = open(source, "rb")
             ^^^^^^^^^^^^^^^^^^
FileNotFoundError: [Errno 2] No such file or directory: 'inventory.xml'
>>> print(type(tree))
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
NameError: name 'tree' is not defined
```

Code 70: Parsing an XML file through `ET.parse()`

- `tree.getroot()` returns the root `Element` of a parsed `ElementTree` object.

```
>>> import xml.etree.ElementTree as ET
>>> tree = ET.parse('inventory.xml')
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File "/opt/anaconda3/lib/python3.11/xml/etree/ElementTree.py", line 1219, in parse
    tree.parse(source, parser)
  File "/opt/anaconda3/lib/python3.11/xml/etree/ElementTree.py", line 570, in parse
    source = open(source, "rb")
             ^^^^^^^^^^^^^^^^^^
FileNotFoundError: [Errno 2] No such file or directory: 'inventory.xml'

>>> root = tree.getroot()
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
NameError: name 'tree' is not defined
>>> print(root.tag)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
NameError: name 'root' is not defined
```

Code 71: Retrieving the root `Element` of an `ElementTree` object using `.getroot()`

(b) `ET.fromstring(s)` parses an XML string and returns the root `Element` object.

```
>>> import xml.etree.ElementTree as ET

>>> xml_data = '<store name="TechHub"><item>Laptop</item></store>'

>>> root = ET.fromstring(xml_data)
```

```
>>> print(type(root))
<class 'xml.etree.ElementTree.Element'>
>>> print(root.tag)
store
```

Code 72: Parsing an XML string through `ET.fromstring()`

- `element.find(tag)` returns the first child `Element` matching the specified tag, or `None` if not found.
- `element.findall(tag)` returns a list of all child `Element`s matching the specified tag.
- `element.get(attr)` retrieves the value of a specified attribute from an `Element`.

```
>>> import xml.etree.ElementTree as ET

>>> xml_data = '''
... <store name="TechHub">
...     <item id="1">Laptop</item>
...     <item id="2">Mouse</item>
... </store>
... '''
>>> root = ET.fromstring(xml_data)

>>> print(root.get('name')) # fetching an attribute value
TechHub

>>> first_item = root.find('item') # getting ONLY the first matching child element
>>> print(first_item.text)
Laptop

>>> all_items = root.findall('item') # retrieving a list of ALL matching child
 ↪  elements
>>> print([item.text for item in all_items])
['Laptop', 'Mouse']
```

Code 73: Retrieving attributes and child `Element`s from a given parent `Element` object using `.find()`, `.findall()`, and `.get()`

6.2 File System & OS Interface

6.2.1 os

Core Concept 6.6: os

The `os` module provides portable tooling for interacting with the underlying operating system. It serves as the standard library's primary interface for file system navigation, directory and file manipulation, and environment configuration management.

(a) **Navigating and Querying the File System**

- `os.getcwd()` returns the absolute path string of the current working directory.

```
>>> import os

>>> current_dir = os.getcwd()
>>> print(current_dir)
/Users/ingyubahng/ibahng-com/computer_science/python
```

Code 74: Querying the active system directory using `os.getcwd()`

- `os.chdir(path)` changes the current working directory to the specified path target.
- `os.listdir(path='.')` returns a list containing the names of the entries within the given directory.

```
>>> import os

>>> entries = os.listdir('.')
>>> print(entries)
['python.synctex.gz', 'math_utils.py', 'python.pdf', 'pythontex-files-python',
 ↪  'python.tex', 'python.fdb_latexmk', 'python.toc', 'session.pkl', 'python.out',
 ↪  '__pycache__', 'data.json', 'sandbox', 'python.pytxcode', 'sensor_data.txt',
 ↪  'python.fls', 'python.log', 'python.aux']
```

Code 75: Listing directory contents through `os.listdir()`

(b) **Managing Files and Directories**

- `os.mkdir(path)` creates a single directory at the specified path.
- `os.makedirs(path)` recursively creates a target directory along with any missing intermediate directories.
- `os.remove(path)` removes a file path target.
- `os.rmdir(path)` removes an empty directory path target.
- `os.rename(src, dst)` renames a file or directory from the source path to the destination path.

```
>>> import os
>>> import shutil

>>> shutil.rmtree('./sandbox', ignore_errors=True) # recursively deletes the entire
 ↪  sandbox directory tree if it exists; this is in order to start with a clean
 ↪  slate

>>> os.makedirs('./sandbox', exist_ok=True)
>>> os.chdir('./sandbox')
>>> print(os.listdir('.'))
[]

>>> os.mkdir('test_dir')
>>> print(os.listdir('.'))
['test_dir']

>>> os.rename('test_dir', 'renamed_dir')
>>> print(os.listdir('.'))
['renamed_dir']

>>> os.makedirs('renamed_dir/sub_parent/child', exist_ok=True)
>>> print(os.listdir('renamed_dir'))
['sub_parent']

>>> with open('renamed_dir/sub_parent/child/temp.txt', 'w') as f:
...     f.write('Clean up test')
...
13
>>> print(os.listdir('renamed_dir/sub_parent/child'))
['temp.txt']

>>> os.remove('renamed_dir/sub_parent/child/temp.txt')
```

```
>>> print(os.listdir('renamed_dir/sub_parent/child'))
[]

>>> os.rmdir('renamed_dir/sub_parent/child')
>>> print(os.listdir('renamed_dir/sub_parent'))
[]

>>> os.chdir('..')
```

Code 76: Demonstrating directory creation, file system manipulation, and selective deletion within a sandboxed environment

(c) **Retrieving Environment Variables**

- `os.environ` maps the current environment keys and values to a dictionary-like object interface.
- `os.getenv(key, default=None)` safe-extracts the configuration value matching a given key without throwing an exception.

```
>>> import os

>>> user_profile = os.getenv('USER', 'default_user')
>>> print(user_profile)
ingyubahng
```

Code 77: Extracting host configuration properties using `os.getenv()`

6.2.2 subprocess

Core Concept 6.7: subprocess

The `subprocess` module provides the standard interface for manipulating processes. In layman's terms, `subprocess` is the module that allows a developer to run terminal commands via Python scripts and subsequently read the results.

(a) **Executing System Commands**

- `subprocess.run(args)` is the primary workhorse function that executes the command described by `args`, waits for command completion, and returns a `CompletedProcess` instance. Security best practices dictate passing commands as a list of strings rather than a single string.

```
>>> import subprocess

>>> result = subprocess.run(['echo', 'Hello from the subshell!'])
>>> print(result) # returncode of 0 means success
CompletedProcess(args=['echo', 'Hello from the subshell!'], returncode=0)
```

Code 78: Executing a basic shell command using `subprocess.run()`

(b) **Capturing Command Output**

- Passing `capture_output=True` alongside `text=True` routes the standard output and standard error streams directly into the returned `CompletedProcess` object as readable string attributes. The absence of `text=True` return byte objects.

```
>>> import subprocess
```

```
2
3 >>> result = subprocess.run(['echo', 'Captured output data'], capture_output=True,
   ↪ text=True)
4
5 >>> print(result)
6 CompletedProcess(args=['echo', 'Captured output data'], returncode=0,
   ↪ stdout='Captured output data\n', stderr='')
```

Code 79: Capturing standard output from an external process

(c) **Enforcing Safe Execution Status**

- Passing `check=True` instructs the function to raise a `subprocess.CalledProcessError` exception if the underlying process exits with a non-zero status code, enforcing strict error boundaries.

```
1 >>> import subprocess
2
3 >>> subprocess.run(['python', '-c', 'import sys; sys.exit(1)'], capture_output=True,
   ↪ text=True) # intentionally exits with an errror, no CalledProcessError
4 CompletedProcess(args=['python', '-c', 'import sys; sys.exit(1)'], returncode=1,
   ↪ stdout='', stderr='')
5
6 >>> subprocess.run(['python', '-c', 'import sys; sys.exit(1)'], check=True,
   ↪ capture_output=True, text=True) # intentionally exits with an error status (1)
7 Traceback (most recent call last):
8   File "<stdin>", line 1, in <module>
9   File "/opt/anaconda3/lib/python3.11/subprocess.py", line 571, in run
10    raise CalledProcessError(retcode, process.args,
11 subprocess.CalledProcessError: Command '['python', '-c', 'import sys; sys.exit(1)']'
   ↪ returned non-zero exit status 1.
```

Code 80: Handling execution failures using `check=True`

6.2.3 sys

Core Concept 6.8: sys

The `sys` module provides access to variables and functions that interact directly with the Python runtime environment. It is the primary tool for querying interpreter details, handling standard input/output streams, processing command-line arguments, and controlling script termination.

(a) **Interpreter and Runtime Information**

- `sys.version` returns a string containing the Python version number alongside additional build information and compiler details.
- `sys.version_info` returns a named tuple detailing the five components of the version number (major, minor, micro, releaselevel, and serial), which is highly useful for programmatic version checking.
- `sys.platform` returns a string identifying the underlying operating system platform—e.g., `'linux'`, `'darwin'`, `'win32'`—on which Python is executing.
- `sys.executable` returns a string providing the absolute path to the executable binary of the Python interpreter currently running the script.

```
1 >>> import sys
2
3 >>> print(sys.version)
```

```
3.11.11 (main, Dec 11 2024, 10:25:04) [Clang 14.0.6 ]
>>> print(sys.version_info)
sys.version_info(major=3, minor=11, micro=11, releaselevel='final', serial=0)
>>> print(sys.platform)
darwin
>>> print(sys.executable)
/opt/anaconda3/bin/python
```

Code 81: Querying host platform and interpreter properties

(b) **Command-Line Arguments**

- `sys.argv` is a list containing the command-line arguments passed to a Python script in the terminal.

```
python sys_arg0 sys_arg1 sys_arg2 ...
```

  By convention, `sys.argv[0]` always holds the name of the script itself, while the sliced list `sys.argv[1:]` isolates the actual arguments provided by the user in the terminal.

(c) **Standard I/O Streams**

- `sys.stdin`, `sys.stdout`, and `sys.stderr` are file objects used by the interpreter for standard input, standard output, and standard errors, respectively. For instance, the built-in `print()` function acts as a convenient wrapper around `sys.stdout.write()`.

```
>>> import sys

>>> # functionally equivalent to print("Standard output message")
>>> sys.stdout.write("Standard output message\n")
Standard output message
24

>>> # emitting an error message that bypasses standard output redirection
>>> sys.stderr.write("Warning: Configuration file missing.\n")
37
```

Code 82: Writing directly to system output and error streams

- Writing directly to `sys.stderr` allows developers to separate error diagnostics and warning messages from the primary program output stream. For instance, the following terminal Python execution script allows errors/warnings and outputs to be outputed into two different files `err.txt` and `out.txt`, respectively.

```
python script.py > out.txt 2> err.txt
```

(d) **Script Termination**

- `sys.exit([arg])` forces the Python interpreter to safely terminate the running script. Passing an integer of `0` implies a successful, expected completion, whereas passing any non-zero integer—commonly `1`—signals an error or abnormal exit condition to the host operating system.

6.2.4 glob

Core Concept 6.9: glob

The `glob` module provides a mechanism for identifying file paths that match a specified pattern according to the rules utilized by the Unix shell. It is the idiomatic standard library tool for pattern-based directory traversal, bulk file identification, and wildcard filtering.

(a) **Pattern Matching and File Retrieval**

- `glob.glob(pathname, *, recursive=False)` returns a populated list of path strings that match the specified `pathname`. The string can contain shell-style wildcards such as `*` (*matches any sequence of characters*), `?` (*matches any single character*), and `[seq]` (*matches any character in* `seq`). When `recursive=True` is passed, the `**` pattern matches zero or more directories and subdirectories recursively.

```
>>> import glob
>>> import os

>>> print(os.getcwd())
/Users/ingyubahng/ibahng-com/computer_science/python

>>> py_files = glob.glob('*.py') # retrieve all Python source files in the active directory
>>> print(py_files)
['math_utils.py']

>>> txt_files = glob.glob('**/*.pkl', recursive=True) # retrieve all text files spanning a nested directory structure
>>> print(txt_files)
['session.pkl', 'pythontex-files-python/pythontex_data.pkl']
```

Code 83: Extracting file paths using shell-style wildcard patterns via `glob.glob()`

(b) **Memory-Efficient Iteration**

- `glob.iglob(pathname, *, recursive=False)` provides the exact same pattern-matching semantics as `glob.glob()`, but returns an iterator instead of a fully realized list. This **deferred execution model** is highly optimal, as it prevents memory exhaustion when parsing directories that contain thousands of files.

```
>>> import glob

>>> csv_iterator = glob.iglob('data/*.csv') # initialize an iterator for CSV files
>>> print(type(csv_iterator))
<class 'generator'>

>>> for filepath in csv_iterator: # consuming the iterator sequentially without bulk memory allocation
...     print(f"Processing: {filepath}")
...
```

Code 84: Generating an iterator for memory-efficient path traversal using `glob.iglob()`

6.3 Mathematics & Randomness

6.3.1 random

Core Concept 6.10: random

The `random` module implements pseudo-random number generators for various distributions. It provides the standard tools for generating randomized data, sampling sequences, and performing probabilistic selections.

(a) **Random Number Generation**

```
>>> import random

>>> for i in range(3):
... print(random.random()) # picks a random number in [0,1)
  File "<stdin>", line 2
    print(random.random()) # picks a random number in [0,1)
    ^^^^^
IndentationError: expected an indented block after 'for' statement on line 1

>>> for i in range(3):
... print(random.randint(1,50)) # only integers and inclusive
  File "<stdin>", line 2
    print(random.randint(1,50)) # only integers and inclusive
    ^^^^^
IndentationError: expected an indented block after 'for' statement on line 1

>>> for i in range(3):
... print(random.uniform(1,50)) # only floats and thus exclusive
  File "<stdin>", line 2
    print(random.uniform(1,50)) # only floats and thus exclusive
    ^^^^^
IndentationError: expected an indented block after 'for' statement on line 1

>>> for i in range(3):
... print(random.randrange(1,12,2)) # picks numbers in steps (default step:1)
  File "<stdin>", line 2
    print(random.randrange(1,12,2)) # picks numbers in steps (default step:1)
    ^^^^^
IndentationError: expected an indented block after 'for' statement on line 1
```

Code 85: Generating constrained pseudo-random numerical values

- `random.random()` returns a floating-point number in the interval $[0.0, 1.0)$.
- `random.randint(a, b)` returns a random integer N such that $a \leq N \leq b$ (*both bounds are inclusive*).
- `random.uniform(a, b)` returns a random floating-point number N such that $a < N < b$.
- `random.randrange(start, stop, step)` returns a randomly selected element from `range(start, stop, step)`.

6.3.2 math

Core Concept 6.11: math

The `math` module provides access to mathematical functions defined by the C standard. It is the primary library for executing standard geometric, logarithmic, and trigonometric calculations.

(a) **Fundamental Mathematical Operations**

```
>>> import math

>>> print(math.sqrt(9))
3.0
>>> print(math.ceil(8.2))
9
>>> print(math.floor(7.9))
7
>>> print(math.fabs(-2.2))
2.2
```

Code 86: applying fundamental numeric operations via the `math` module

- `math.sqrt(x)` returns the square root of x.
- `math.ceil(x)` returns the ceiling of x, the smallest integer greater than or equal to x.
- `math.floor(x)` returns the floor of x, the largest integer less than or equal to x.
- `math.fabs(x)` returns the absolute value of x as a float.

(b) **Mathematical Constants**

```
>>> import math

>>> r = 5.0
>>> area = math.pi * (r ** 2)
>>> print(area)
78.53981633974483
```

Code 87: Calculating circular area utilizing `math.pi`

- The module provides access to immutable standard mathematical constants, such as `math.pi` and `math.e`.

6.4 Time & Scheduling

6.4.1 datetime

Core Concept 6.12: datetime

The `datetime` module supplies classes for manipulating dates and times. It provides tools for date arithmetic, time formatting, and calendar data extraction through its core classes, which are conventionally imported using the `from datetime import ...` syntax.

(a) **Date /** `date`

```
>>> from datetime import date

>>> today = date.today() # current local date
>>> d = date(2026, 5, 29) # construct a specific date

>>> d.year
2026
>>> d.month
5
>>> d.day
29
>>> d.weekday()
4
>>> d.isoformat()
'2026-05-29'
>>> d.strftime("%d %B %Y")
'29 May 2026'
```

Code 88: Demonstrating `date` class functionalities and attribute extraction

- The `date` class represents a localized calendar date (*year, month, and day*) without time or time zone considerations. Instances allow for direct attribute extraction and ISO 8601 string formatting via `.isoformat()`.
- `obj.strftime(format)` converts a temporal object into a formatted human-readable string—such as `%d %B %Y`—whereas `datetime.strptime(string, format)` parses a formatted string back into a temporal object.

  When parsing or formatting dates in Python, you use specific percentage-prefixed format codes where each letter acts as a unique placeholder for a distinct temporal component. This system allows you to build highly customized date and time strings by freely combining these distinct codes with regular text and punctuation.

Code	Meaning	Example
`%Y`	4-digit year	`2026`
`%m`	Zero-padded month	`05`
`%d`	Zero-padded day	`29`
`%H`	Hour, 24hr	`14`
`%M`	Minute	`30`
`%S`	Second	`00`
`%A`	Full weekday name	`Friday`
`%B`	Full month name	`May`
`%f`	Microseconds	`000000`

Table 1: Standard format codes for parsing and formatting `datetime` objects.

(b) **Datetime** / `datetime`

```
>>> from datetime import datetime

>>> now = datetime.now() # current local date and time
```

```
>>> utc = datetime.utcnow() # current UTC time (naive)

>>> dt = datetime(2026, 5, 29, 14, 30, 0)

>>> dt.date() # extract date portion → date(2026, 5, 29)
datetime.date(2026, 5, 29)
>>> dt.time() # extract time portion → time(14, 30, 0)
datetime.time(14, 30)
>>> dt.isoformat() # '2026-05-29T14:30:00'
'2026-05-29T14:30:00'
>>> dt.strftime("%Y-%m-%d %H:%M") # '2026-05-29 14:30'
'2026-05-29 14:30'

>>> datetime.strptime("29/05/2026", "%d/%m/%Y") # Parsing a string into a datetime
 ↪  object
datetime.datetime(2026, 5, 29, 0, 0)
```

Code 89: Extracting and formatting date and time components from a `datetime` object

- `datetime.now()` returns the current local date and time, while `datetime.utcnow()` returns the current naive UTC time. Both are frequently used to initialize a temporal baseline.
- A `datetime` instance encapsulates both date and time. You can isolate these components using the `.date()` and `.time()` methods, which return their respective constituent objects.
- Similar to the `date` class, `datetime` objects support standardized output through `.isoformat()` and can be manipulated using the `strptime()` and `strftime()` string formatting functions.

(c) **Timedelta /** `timedelta`

```
>>> from datetime import timedelta

>>> delta = timedelta(days=7, hours=3)

>>> date.today() + delta # one week and 3 hours from now
datetime.date(2026, 6, 5)
>>> date.today() - delta # one week and 3 hours ago
datetime.date(2026, 5, 22)

>>> d1 = datetime(2026, 1, 1)
>>> d2 = datetime(2026, 5, 29)
>>> diff = d2 - d1
>>> diff.days # 148
148
>>> diff.total_seconds() # 12787200.0
12787200.0
```

Code 90: Executing temporal arithmetic and calculating time differences using `timedelta`

- The `timedelta` class represents a duration or the mathematical difference between two dates or times. It enables straightforward temporal arithmetic, allowing you to add or subtract specific time intervals from existing `date` or `datetime` objects.
- Subtracting two `datetime` instances automatically yields a `timedelta` object, which can then be queried for its total duration using attributes like `.days` or the `.total_seconds()` method.

6.4.2 time

Core Concept 6.13: time

The `time` module provides time-related functions that primarily deal with system clocks and low-level time tracking, rather than calendar dates.

(a) **System Time and Execution Control**

- The **Epoch** is the point where time mathematically begins for Unix systems: January 1, 1970, 00:00:00 (UTC).
- `time.time()` returns the current time as a floating-point number expressed in seconds since the epoch.
- `time.ctime(secs)` converts a time expressed in seconds since the epoch into a human-readable string.
- `time.perf_counter()` returns a float value from a clock with the highest available resolution, making it ideal for benchmarking code execution.
- `time.sleep(secs)` pauses the execution of the current thread for the given number of seconds.

```
>>> import time

>>> start_time = time.perf_counter()

>>> print(f"Seconds since Epoch: {time.time()}")
Seconds since Epoch: 1780065580.9174662
>>> print(f"Readable String: {time.ctime()}")
Readable String: Fri May 29 23:39:40 2026

>>> # Suspend execution for 0.1 seconds
>>> time.sleep(0.1)

>>> end_time = time.perf_counter()
>>> print(f"Execution took: {end_time - start_time:.4f} seconds")
Execution took: 0.1034 seconds
```

Code 91: Querying system time and benchmarking execution duration

6.5 Concurrency & Data Structures

6.5.1 queue

Core Concept 6.14: queue

6.6 Regular Expressions

6.6.1 re

Core Concept 6.15: re

re https://docs.python.org/3/library/re.html

6.6.2 SWE Utilities

6.6.3 logging

Core Concept 6.16: logging

6.6.4 abc

Core Concept 6.17: abc

abc https://docs.python.org/3/library/abc.html

7 Object Oriented Programming

7.1 Classes

Core Concept 7.1: Classes, Attributes, and Methods

A `class` is a blueprint for creating objects that encapsulates data and behavior into a cohesive construct, enabling modular code organization and reusability.

```
>>> class BankAccount:
...     bank = "Chase" # class attribute
...     def __init__(self, owner, balance=0):
...         self.owner = owner # instance attribute
...         self.balance = balance # instance attribute
...     def deposit(self, amount): #method
...         self.balance += amount
...         return self.balance
...     def withdraw(self, amount):
...         if amount <= self.balance:
...             self.balance -= amount
...             return self.balance
...         return "Insufficient funds"
...
>>> account = BankAccount("Alice", 100)
>>> print(account.deposit(50))
150
>>> print(account.withdraw(30))
120
>>> print(account.balance)
120
```

Code 92: `class` instantiation, attributes, and methods

The `__init__()` method serves as a special constructor that initializes new class instances, executing automatically upon object instantiation. The `self` parameter references the specific instance being created or manipulated, providing access to that instance's attributes and methods.

Attributes represent the data stored within a class and are categorized into two distinct types.

- **Class attributes** are shared uniformly across all instances of a class. In the `BankAccount` example, `bank = "Chase"` constitutes a class attribute accessible to all `BankAccount` objects.

- **Instance attributes** are unique to each individual object. Attributes such as `self.owner` and `self.balance` must be explicitly provided during instantiation, as demonstrated by `account = BankAccount("Alice", 100)`.

Methods are functions defined within a class namespace that operate on instance data. These functions are bound to the class and accessible only through class instances. In the example above, `deposit()` and `withdraw()` exemplify methods that represent the behavioral logic of the `BankAccount` class, specifically the `self.balance` attribute.

Core Concept 7.2: Setters & Getters

Setters and **getters** are methods that control access to an object's attributes outside of the initialization of the class. These methods of control can be written in one of two ways.

(a) **Explicit Method**: The setter and getter are just written as plain functions.

```
>>> class Example:
...     def setName(self, n): # setter
...         self.name = n
...     def getName(self): # getter
...         return self.name
...
>>> p1 = Example()
>>> p1.setName("John")
>>> print(p1.getName())
John
```

Code 93: Explicit setter and getter method

(b) **Decorator Method**: The setter and getter methods are decorated with the `@property` and `@<attribute>.setter` annotations.

By convention, attributes prefixed with a single underscore in the format of `_attribute` indicate internal implementation details that should not be accessed directly, though Python does not enforce true privacy.

```
>>> class PythonicExample:
...     def __init__(self, name):
...         self._name = name  # Protected backing attribute
...     @property
...     def name(self):  # Getter decorator
...         return self._name
...     @name.setter
...     def name(self, n):  # Setter decorator
...         if not n:
...             raise ValueError("Name cannot be empty")
...         self._name = n
...
>>> p2 = PythonicExample("John")
>>> p2.name = "Sally"  # Intercepted by the setter seamlessly
>>> print(p2.name)     # Intercepted by the getter seamlessly
Sally
```

Code 94: Decorator setter and getter method

The difference is how Python handles attribute access under the hood. The explicit method relies on traditional function calls like `p1.setName()`, which creates a nightmare when trying to convert all `p1.name = <name>`

lines for all class instances to the `p1.setName()` method, resulting in every file interacting with that variable having to be refactored.

Conversely, the decorator method hooks directly into Python's descriptor protocol such as `p2.name = "Sally"` (*setter*) and `p2.name` (*getter*) in the above example. This wraps the underlying functions into a property descriptor object, allowing outside code to maintain the clean, direct assignment syntax of `p2.name = "Sally"`.

The decorator method is simply the undisputed industry standard *way to handle setters and getters because it gives the developer the power to add data validation, transformation, or read-only access control in the background with ease. For instance, omitting the matching setter to a property getter creates a strict read-only functionality for an attribute.*

```
>>> class ReadOnlyExample:
...     def __init__(self, name):
...         self._name = name  # Protected backing attribute
...     @property
...     def name(self):  # Getter decorator
...         return self._name
...     # Note: The .setter decorator is intentionally omitted to make it read-only
...
>>> p3 = ReadOnlyExample("John")
>>> print(p3.name)      # Read access works perfectly via the getter
John

>>> # Attempting to modify the attribute will now throw an AttributeError
>>> p3.name = "Sally"
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
AttributeError: property 'name' of 'ReadOnlyExample' object has no setter
```

Code 95: Setter omission creating a read-only functionality for an attribute

7.2 The Four Pillars of Object Oriented Programming

Core Concept 7.3: Encapsulation

Encapsulation is the practice of bundling data attributes and their associated methods into a cohesive class unit while restricting direct external access through access control mechanisms. This design principle protects internal state integrity by exposing controlled interfaces rather than allowing direct manipulation of object internals.

```
>>> class BankAccount:
...     def __init__(self, owner, initial_deposit):
...         self.owner = owner
...         self.__balance = initial_deposit
...     @property
...     def balance(self):
...         return f"{self.__balance:,.2f}"
...     def display_balance(self):
...         return f"{self.__balance:,.2f}"
...
>>> account = BankAccount("Ingyu", 1000)
>>> print(account.balance) # getter method
1,000.00
>>> print(account.display_balance()) # explicit function
1,000.00
>>> print(account._BankAccount__balance) # name mangling
1000
```

```
18
19 >>> print(account.__balance) # private attributes cannot be directly accessed
20 Traceback (most recent call last):
21   File "<stdin>", line 1, in <module>
22 AttributeError: 'BankAccount' object has no attribute '__balance'
```

Code 96: Encapsulation and `class` private attributes using `__attribute` syntax

The double underscore prefix (`__attribute`) designates an attribute as private, triggering Python's name mangling mechanism that converts the attribute name to `_ClassName__attribute`. While this prevents direct external access via the original attribute name, the value remains retrievable through properly designed accessor methods (*getters*), explicit retrieval functions, or by explicitly invoking the mangled name syntax—though the latter circumvents the intended encapsulation and is considered poor practice.

Core Concept 7.4: Inheritance

Inheritance is a fundamental object-oriented mechanism whereby a child class (*subclass*) acquires the attributes and methods of a parent class (*superclass*), facilitating code reuse and enabling the hierarchical extension of functionality without redundancy.

```
1  >>> class Animal:
2  ...     def __init__(self, name):
3  ...         self.name = name
4  ...     def eat(self):
5  ...         return f"{self.name} is eating."
6  ...     def speak(self):
7  ...         return f"{self.name} makes a generic sound."
8  ...
9  >>> class Dog(Animal):
10 ...     def speak(self):
11 ...         return f"{self.name} barks: Woof!"
12 ...
13 >>> my_dog = Dog("Buddy")
14 >>> print(my_dog.eat())
15 Buddy is eating.
16 >>> print(my_dog.speak())
17 Buddy barks: Woof!
```

Code 97: Inheritance of parent `Animal` by child `Dog`

In the above code block, the `Dog` class inherits all attributes and methods from the `Animal` parent class. When a child class defines a method with an identical name to a parent class method—such as `speak()` in this example—the child implementation completely overrides the parent version, a behavior known as **method overriding**.

In addition to the simple inheritance shown above, the `super()` function allows the user to return a proxy object that delegates method calls to the parent class, enabling controlled inheritance and method extension. This mechanism serves two primary purposes.

- **Constructor Extension**: Child class constructors frequently require both parent initialization logic and child-specific setup. The expression `super().__init__()` delegates baseline initialization to the parent constructor, ensuring proper attribute inheritance while permitting the addition of child-specific attributes.
- **Method Extension**: While method overriding replaces parent functionality entirely,

`super().method_name()` allows child classes to invoke parent method logic before or after executing child-specific operations, thereby extending rather than replacing inherited behavior.

```
>>> class Vehicle:
...     def __init__(self, brand, model):
...         self.brand = brand
...         self.model = model
...     def description(self):
...         return f"{self.brand} {self.model}"
...
>>> class Car(Vehicle):
...     def __init__(self, brand, model, doors):
...         super().__init__(brand, model)
...         self.doors = doors
...     def description(self):
...         base_info = super().description()
...         return f"{base_info} with {self.doors} doors"
...
>>> car = Car("Toyota", "Camry", 4)
>>> print(car.description())
Toyota Camry with 4 doors
```

Code 98: Method extension using `super()`

Core Concept 7.5: Polymorphism

Polymorphism is the capacity of different classes to implement identically named methods, enabling a single unified interface to operate on diverse object types while invoking class-specific behavior. This principle allows functions to process heterogeneous objects uniformly, with runtime behavior determined by each object's concrete class implementation.

```
>>> class MasterCard:
...     def process_payment(self, amount):
...         return f"Processing ${amount:.2f} via MasterCard networks."
...
>>> class PayPal:
...     def process_payment(self, amount):
...         return f"Routing ${amount:.2f} through PayPal wallets."
...
>>> class Bitcoin:
...     def process_payment(self, amount):
...         return f"Broadcasting ${amount:.2f} to blockchain."
...
>>> class ApplePay:
...     def process_payment(self, amount):
...         return f"Authenticating ${amount:.2f} via Apple."
...
>>> def checkout_gate(payment_processor, total):
...     print(payment_processor.process_payment(total))
...
>>> visa, wallet, crypto, mobile = MasterCard(), PayPal(), Bitcoin(), ApplePay()

>>> checkout_gate(visa, 150.00)
Processing $150.00 via MasterCard networks.
>>> checkout_gate(wallet, 150.00)
Routing $150.00 through PayPal wallets.
>>> checkout_gate(crypto, 150.00)
Broadcasting $150.00 to blockchain.
>>> checkout_gate(mobile, 150.00)
```

```
Authenticating $150.00 via Apple.
```

Code 99: Polymorphism and identically named method `process_payment` executing different actions based on paired `class`

Each payment processor class implements an identically named `process_payment()` method with class-specific logic. The `checkout_gate()` function accepts any payment processor object and invokes its `process_payment()` method, demonstrating polymorphic behavior: the same function call produces different outputs depending on the runtime type of the object passed as an argument. This design enables extensibility—new payment processor classes can be added without modifying the `checkout_gate()` function, provided they implement the `process_payment()` interface.

Core Concept 7.6: Abstraction

Abstraction is a fundamental architectural principle that conceals complex, low-level implementation details behind simplified interfaces, restricting external entities to interacting only with an object's external behavior rather than its inner mechanisms, a constraint often realized through **encapsulation** (*Core Concept 7.3*), **inheritance** (*Core Concept 7.4*), and **polymorphism** (*Core Concept 7.5*).

Explicit abstraction—facilitated by Python's `abc` library—enforces a mandatory structural baseline for all derived subclasses. Furthermore, it prohibits direct instantiation of the abstract base class, ensuring access is only mediated through its child class implementations.

```
>>> from abc import ABC, abstractmethod
>>> class PrinterBlueprint(ABC):
...     @abstractmethod
...     def print_page(self, text):
...         pass
...
>>> class OfficePrinter(PrinterBlueprint): # Matches the blueprint EXACTLY
...     def print_page(self, text):
...         return f"Printing office document: {text}"
...
>>> class BrokenPrinter(PrinterBlueprint): # Flaw: accidental rename
...     def output_text(self, text):
...         return f"Printing: {text}"
...
>>> printer_1 = OfficePrinter() # success
>>> printer_2 = BrokenPrinter() # renamed mandatory method
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: Can't instantiate abstract class BrokenPrinter with abstract method print_page
>>> printer_3 = PrinterBlueprint() # can't call the abstract parent class
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: Can't instantiate abstract class PrinterBlueprint with abstract method
 ↪ print_page
```

Code 100: Explicit abstraction using the `abc` module

As demonstrated above, the instantiation of the `BrokenPrinter` class fails because it neglects to implement the required `print_page()` method prescribed by its abstract base class, `PrinterBlueprint`. Conversely, the `OfficePrinter` instantiation succeeds because it strictly adheres to the same method signature outlined within `PrinterBlueprint`.

8 Error Handling & Debugging

8.1 Exceptions Handling

Core Concept 8.1: Exceptions

Exceptions are runtime errors that cause the abnormal termination of a program and its resources. Examples include `ZeroDivisionError`, which occurs when attempting to divide a value by zero, and `IndexError`, which arises when accessing an element within a collection (*such as a list or array*) using an invalid index.

Core Concept 8.2: Custom Exceptions

Developers can define **custom exceptions** by subclassing the base `Exception` class, which can subsequently be triggered using the `raise` statement.

```
>>> class InvalidAgeError(Exception):
...     pass # no need for constructor
...
>>> def verify_age(age):
...     if age < 0:
...         raise InvalidAgeError(f"Age cannot be negative. Received: {age}")
...     print(f"Age {age} successfully verified.")
...
>>> try:
...     print("System: Verifying user age...")
...     verify_age(-5)
...
... except InvalidAgeError as e:
...     print(f"Error Type: {type(e).__name__}")
...     print(f"Error Message: {e}")
...
System: Verifying user age...
Error Type: InvalidAgeError
Error Message: Age cannot be negative. Received: -5
```

Code 101: Custom `InvalidAgeError` exception

As demonstrated above, the custom exception subclass does not require an explicit constructor. It inherits the constructor from the base `Exception` class, which is already designed to accept arguments—specifically, a string representing an error message.

Core Concept 8.3: try, except, & finally

The `try`, `except`, and `finally` structure provides a mechanism to manage and handle exceptions that arise during the execution of a designated block of code.

```
>>> try:
...     print("Attempting the calculation...")
...     result = 10 / 0
...
...     print(f"The result is {result}") # This line will be skipped because the error
    ↪ happens above
...
... except ZeroDivisionError:
```

```
...     print("Error: You cannot divide a number by zero!")
... finally:
...     print("Cleanup: The division operation has finished.")
...
Attempting the calculation...
Error: You cannot divide a number by zero!
Cleanup: The division operation has finished.
```

Code 102: Exception handling with `try` / `except` / `finally` block

As demonstrated above, the code within the `try` block executes first. If a specified exception—in this case, a `ZeroDivisionError`—is raised, the runtime environment prevents premature termination and transfers control to the `except` block. The `finally` block subsequently executes regardless of whether the `try` block succeeded or failed. Consequently, the `finally` block is typically reserved for essential cleanup operations, such as terminating database connections or closing files.

If a specific exception type is not explicitly defined, a generic `except` block will default to catching all potential exceptions. This approach is generally considered **poor practice**, as it can obscure unrelated bugs and lead to unpredictable program behavior, particularly in complex software.

```
>>> try:
...     print("Attempting the calculation...")
...     result = 10 / 0
...
...     print(f"The result is {result}") # This line will be skipped because the error happens above
...
... except: # catches ALL errors
...     print("Error: You cannot divide a number by zero!")
... finally:
...     print("Cleanup: The division operation has finished.")
...
Attempting the calculation...
Error: You cannot divide a number by zero!
Cleanup: The division operation has finished.
```

Code 103: Catching all exceptions with generic `except:` header; *poor practice*

A more robust practice involves implementing multiple `except` headers to handle distinct exception types individually. Utilizing the base `Exception` class as a fallback, paired with `as e` to capture the error object, allows the program to safely identify and log unexpected exceptions while maintaining application stability.

```
>>> try:
...     user_input = "Hello"
...     number = int(user_input)
...
... except ValueError as e:
...     print(f"The message is: {e}")
...     print(f"The error name is: {type(e).__name__}")
... except Exception as e: # This is a fallback that catches anything else and prints its name
...     print(f"An unexpected {type(e).__name__} occurred: {e}")
...
The message is: invalid literal for int() with base 10: 'Hello'
The error name is: ValueError
```

Code 104: Multiple `except` blocks

8.2 Logging

Core Concept 8.4: Logging

Logging is a systematic mechanism for tracking events, errors, and state changes that occur during software execution. While standard `print()` statements are frequently utilized for *preliminary* debugging, the built-in `logging` library provides a more robust framework suitable for larger applications. It allows developers to **categorize outputed messages by severity**.

The logging framework categorizes events into five primary severity levels, listed in increasing order of critical importance: `DEBUG`, `INFO`, `WARNING`, `ERROR`, and `CRITICAL`. By establishing a minimum severity threshold, the runtime environment can automatically filter out lower-level messages, ensuring that only pertinent information is captured and recorded.

```
>>> import sys
>>> import logging

>>> logging.basicConfig(level=logging.WARNING, format='%(levelname)s: %(message)s',
 ↪  stream=sys.stdout, force=True) # force=True and stream=sys.stdout is needed to output
 ↪  the logs within this pyconsole environment

>>> def divide_values(a, b):
...     # This will be suppressed because DEBUG is lower than WARNING
...     logging.debug(f"Attempting to divide {a} by {b}.")
...     if b == 0:
...         # This will be captured because ERROR is higher than WARNING
...         logging.error("Division by zero attempted.")
...         return None
...     result = a / b
...     # This will be suppressed because INFO is lower than WARNING
...     logging.info(f"Division successful. Result is {result}.")
...     return result
...
>>> logging.warning("System is operating with minimal resources.")
WARNING: System is operating with minimal resources.
>>> divide_values(10, 0)
ERROR: Division by zero attempted.
```

Code 105: `logging` module usage at `WARNING` level

To retain log records for historical reference, a destination file can be specified using the `filename` parameter within the `logging.basicConfig()` function, which will route all subsequent log entries to that designated file.

8.3 Assertion

Core Concept 8.5: Assertions

An **assertion** is a programmatic debugging aid that functions as an internal sanity check. It allows developers to explicitly declare a condition that must evaluate to true at a specific point during execution. If the condition holds true, the program proceeds normally without interruption. However, if the condition evaluates to false, the runtime environment immediately halts execution and raises an `AssertionError`.

In Python, assertions are implemented utilizing the `assert` keyword, which evaluates a boolean expression and can optionally include a diagnostic error message.

```
>>> def apply_discount(original_price, discount_rate):
...     # Assert that the parameters fall within logically valid bounds
...     assert original_price > 0, "The original price must be strictly positive."
...     assert 0 <= discount_rate <= 1, "The discount rate must be between 0 and 1."
...     return original_price * (1 - discount_rate)
...
>>> final_price = apply_discount(100.0, 0.2) # This will execute successfully
>>> print(f"Discounted price: {final_price}")
Discounted price: 80.0

>>> apply_discount(-50.0, 0.2) # This will immediately trigger an AssertionError
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File "<stdin>", line 3, in apply_discount
AssertionError: The original price must be strictly positive.
```

Code 106: Internal logic verification through `assert` keyword

In practice, assertions are designed to **verify internal logic** and identify states that the developer considers computationally impossible, primarily used as tools for development and testing phases.

9 File Input/Output and Data Handling

Core Concept 9.1: Files

A **file** is a continuous, one-dimensional **stream** of data—either plain text characters or raw binary bytes—that an application can sequentially read from or write to during execution.

Core Concept 9.2: File Input/Output

To interact with a file, a program must request a **file object** from the operating system. This object acts as a temporary communication bridge between the application's volatile memory and the persistent storage drive. The fundamental lifecycle of programmatic File I/O strictly adheres to three sequential phases.

(a) **Open:** Establishing the stream connection and declaring the intended access mode.

- `'r'` / **read**: Opens a file exclusively for reading, positioning the stream at the beginning of the document. If the specified file does not exist, the runtime raises a `FileNotFoundError`.
- `'w'` / **write**: If the file already exists, its **existing contents are immediately truncated** (*completely overwritten*) before new data is introduced. If it does not exist, a new file is generated.
- `'a'` / **append**: Opens a file for writing but **preserves existing data** by positioning the stream at the very end of the document. Any newly transmitted data is appended to the conclusion of the file. If the file does not exist, a new one is created.
- `'x'` / **exclusive creation**: Opens a file strictly for writing, functioning as a safeguard against data loss. **The file must not exist prior to execution**; if it does, a `FileExistsError` is raised, intentionally aborting the operation.

If working with binary files, simply append `'b'` to the end: `'rb'`, `'wb'`, `'ab'`, `'xb'`

(b) **Process:** Transmitting data through the stream via read or write operations.

(c) **Close:** Severing the connection to flush internal memory.

Modern Python development heavily relies on the `with` statement, which acts as a context manager. This structure guarantees that the file stream is **automatically and safely closed once the nested block of code finishes execution**, even if an unexpected exception is raised during the processing phase.

```
>>> with open("sensor_data.txt", "w") as file_stream:
...     _ = file_stream.write("timestamp: 001, status: active\n")
...     _ = file_stream.write("timestamp: 002, status: active\n")
...     # file stream is automatically closed upon exiting this block
...
>>> with open("sensor_data.txt", "r") as file_stream:
...     content = file_stream.read()
...     print(content)
...
timestamp: 001, status: active
timestamp: 002, status: active
```

Code 107: File opening and closing through `with` statement

Core Concept 9.3: Serialization and Pickling

In data handling, **serialization** is the process of converting an in-memory object into a byte stream suitable for storage or transfer. In Python, this implemented via the `pickle` module. The inverse operation—reconstructing a Python object from a byte stream—is known as **deserialization** (*or unpickling*).

Pickling is highly advantageous when working with complex, custom data structures that cannot be easily represented in standard, human-readable text formats like JSON or CSV.

The `pickle` API primarily relies on two functions for file interactions:

- `pickle.dump(obj, file)` serializes the target object and writes the resulting bytes directly to an open binary file stream.
- `pickle.load(file)` reads a binary file stream containing a serialized byte string and reconstructs the original Python object.

```
>>> import pickle

>>> session_data = { # complex data structure
...     "user_id": 1042,
...     "permissions": ["admin", "developer"],
...     "matrix_weights": (0.85, 0.12, 0.94)
... }

>>> with open("session.pkl", "wb") as binary_stream: # serializing data
...     pickle.dump(session_data, binary_stream)
...
>>> with open("session.pkl", "rb") as binary_stream: # deserializing data
...     retrieved_data = pickle.load(binary_stream)
...
>>> print(f"Data Type: {type(retrieved_data)}")
Data Type: <class 'dict'>
>>> print(f"Content: {retrieved_data}")
Content: {'user_id': 1042, 'permissions': ['admin', 'developer'], 'matrix_weights': (0.85,
 ↪  0.12, 0.94)}
```

Code 108: Pickle object serialization through `pickle` module commands `.dump()` and `.load()`

It is important to keep in mind that deserializing a corrupted or maliciously constructed byte stream can trigger arbitrary code execution within the runtime environment. Consequently, objects should never be unpickled from untrusted or unauthenticated external sources.

10 Regular Expression

Core Concept 10.1: Regular Expression

Regular expressions (regex) are character sequences or structural patterns that, when evaluated by the `re` module, enable developers to perform substring searches and pattern validation against a given string.

Core Concept 10.2: Sequence Characters

A regular expression pattern may be composed of raw literal characters, such as `a` or `b`, or **sequence characters**, which are predefined escape sequences that each represent a distinct subset of the full Unicode character space.

- `\d` / **Digits**: Matches any numeric digit character from 0 to 9.
- `\D` / **Non Digits**: Matches any character that is not a numeric digit.
- `\s` / **White Space**: Matches whitespace characters such as spaces, tabs, and newline characters.
- `\S` / **Non White Space**: Matches any character that is not a whitespace character.
- `\w` / **Alphanumeric**: Matches alphanumeric characters and underscores.
- `\W` / **Non Alphanumeric**: Matches any character that is not alphanumeric or an underscore.
- `\b` / **Word Boundary**: Matches positions surrounding complete words.
- `\A` / **Start of String**: Restricts the search to the beginning of the string.
- `\Z` / **End of String**: Restricts the search to the end of the string.

Core Concept 10.3: Core Regular Expression Functions

The `re` module exposes several core functions that make up the primary toolset for applying regular expressions against a string. Each function differs in its scope of search and the form of its return value, as demonstrated in the following list.

(a) The `re.search()` function scans the entire string and returns a **match object** corresponding to the first occurrence of the pattern, or `None` if no match is found; the exact matched substring is then retrievable via the `.group()` method on that object.

```
>>> import re
>>> string = "Take 1 up One 23 idea. One idea 45 at a time"

>>> result = re.search(r'O\w',string)
>>> print(result.group())
On
```

Code 109: `re.search()` paired with `.group()` returning the first matching substring

(b) The `re.findall()` function traverses the full string and returns a list of all non-overlapping matches, making it the appropriate choice when multiple occurrences are expected. This does not need to paired with the `.group` method.

```
>>> import re
>>> string = "Take 1 up One 23 idea. One idea 45 at a time"

>>> result = re.findall(r'O\w\w',string)
>>> print(result)
['One', 'One']
```

Code 110: `re.findall()` returning a list of matching substrings

(c) The `re.match()` function—paired with the `.group()` method—is more restrictive, as it anchors its search **exclusively to the beginning of the string** and returns `None` for any pattern not satisfied at that position.

```
>>> import re
>>> string = "Take 1 up One 23 idea. One idea 45 at a time"

>>> result = re.match('T\w\w',string)
>>> print(result.group())
Tak
```

Code 111: `re.match()` paired with `.group()` returning the matching substring at the beginning of reference string

(d) The `re.sub()` performs an in-place substitution of all matched substrings with a specified replacement

```
>>> import re
>>> string = "Take 1 up One 23 idea. One idea 45 at a time"

>>> result = re.sub(r'One','Two',string)
>>> print(result)
Take 1 up Two 23 idea. Two idea 45 at a time
```

Code 112: `re.sub()` replacing all matched substrings

(e) The `re.split()` partitions the string into a list of substrings at each position where the pattern is matched.

```
>>> import re
>>> string = "Take 1 up One 23 idea. One idea 45 at a time"

>>> result = re.split(r'\d+',string)
>>> print(result)
['Take ', ' up One ', ' idea. One idea ', ' at a time']
```

Code 113: `re.split()` spliting given string at all match substring positions

Core Concept 10.4: Quantifiers

Quantifiers extend the expressive power of a regex pattern by specifying the number of times a preceding character or sequence character must occur in order for a match to be satisfied. They may enforce an exact

repetition count or define a permissible range.

(a) The `{m}` quantifier constrains the preceding token to exactly m repetitions, whereas `{m,n}` defines an inclusive range between a minimum of m and a maximum of n repetitions.

```
>>> import re
>>> string = "Take 1 up One 23 idea.One idea 45 at a Time"

>>> result = re.findall(r'O\w{1}',string) #{m} exactly m repetition
>>> print(result)
['On', 'On']

>>> result = re.findall(r'O\w{1,2}',string) #{m,n} m minimum, n maximum repetitions
>>> print(result)
['One', 'One']
```

Code 114: Regexes using quantifiers

(b) The `+` quantifier requires one or more occurrences, `*` permits zero or more, and `?` allows either zero or one, effectively rendering the preceding token optional.

```
>>> import re
>>> string = "Take 1 up One 23 idea.One idea 45 at a Time"

>>> result = re.findall(r'O\w+',string) #+ one or more repetitions
>>> print(result)
['One', 'One']

>>> result = re.findall(r'O\w*',string) #* zero or more repetitions
>>> print(result)
['One', 'One']

>>> result = re.findall(r'O\w?',string) #? zero or one repetitions
>>> print(result)
['On', 'On']
```

Code 115: Regexes using quantifiers

Mind that quantifiers are greedy by default, meaning the engine will consume as many characters as possible while still producing an overall match. For instance, the `1,2` quantifier in Code 114 yields a list of `'One'` strings rather than `'On'`, as the engine greedily consumes the maximum number of permitted repetitions.

Core Concept 10.5: Special Characters

In addition to **sequence characters** (*Core Concept 10.2*) and **quantifiers** (*Core Concept 10.2*), the `re` module supports a set of **special characters** that modify the structural behaviour of a pattern, such as constraining its position within the string, escaping reserved metacharacters, or expressing logical alternation between sub-patterns.

(a) The unescaped dot `.` serves as a wildcard, matching any single character except a newline; when a literal dot is intended, it must be escaped as `\.` to suppress its metacharacter interpretation. This escaping convention applies universally to all reserved metacharacters within the `re` module.

```
>>> import re
>>> string = "I live .. like cats oh yeah"

```

```
>>> result = re.search(r'l..e',string)
>>> print(result.group())
live

>>> result = re.search(r'\.\.',string)

>>> print(result.group())
..
```

Code 116: `.` and `\.` special characters

(b) The anchors `^` and `$` constrain a match to the start and end of the string respectively, analogously to `\A` and `\Z`.

```
>>> import re
>>> string = "I live .. like cats oh yeah"

>>> result = re.search(r'^I...',string) # beginning
>>> print(result.group())
I li

>>> result = re.search(r'...h$',string) # end
>>> print(result.group())
yeah
```

Code 117: Regex matching at the beginning and end of given string using `^` and `$`

(c) Character classes, denoted by `[...]`, define an explicit set of permissible characters at a given position, while the negated form `[^...]` inverts this constraint to match any character outside the specified set.

```
>>> import re
>>> string = "I live .. like cats oh yeah"

>>> result = re.search(r'[aeiou]',string)
>>> print(result.group())
i

>>> result = re.search(r'[^I li]',string)
>>> print(result.group())
v
```

Code 118: Defining sets of permissible characters in a regex

(d) The alternation operator `(R|S)` evaluates two or more sub-expressions and returns a match if either is satisfied, providing a concise mechanism for expressing logical disjunction within a pattern.

```
>>> import re
>>> string = "I live .. like cats oh yeah"

>>> result = re.search(r'(cats|dogs)',string) #(R|S) specifies multiple regular
 ↪  expressions to match; either R OR S
>>> print(result.group())
cats
```

Code 119: Alternation operator `(R|S)` in regex matching

11 Threading

Core Concept 11.1: Threading & Global Interpreter Lock (GIL)

So far, all previous coding examples shown have been **single-threaded**, meaning that line 1 finishes before moving onto line 2, line 3, and so on. However, with the `threading` module, a developer is able to execute several concurrent operations within a single **multi-threaded** process in a practice called **threading**.

Core Concept 11.2: Main Thread

The main thread that is always running is called the `MainThread`, which can be retrieved through the `threading` module.

```
>>> import threading

>>> print('Current thread that is running: {}'.format(threading.current_thread().name))
Current thread that is running: MainThread

>>> if threading.current_thread() == threading.main_thread():
...     print('Main Thread')
...
Main Thread
```

Code 120: `MainThread` confirmation

11.1 Threading Methods

Core Concept 11.3: Threading Through Functions

Threading in its most elementary form is invoked by passing a callable into the `Thread` object, which instantiates a new thread of execution. The `Thread` constructor accepts two principal parameters: `target`, which holds a reference to the function object in its non-invoked state, and `args`, which supplies the positional arguments to that function as a **tuple**.

```
>>> import threading

>>> def displayNumbers(count):
...     i = 0
...     print(threading.current_thread().name)
...     while i <= count:
...         print(i)
...         i += 1
...
>>> print(threading.current_thread().name)
MainThread

>>> # notice the args value below is a tuple
>>> t = threading.Thread(target=displayNumbers, args = (5,), name =
 ↪  'displayNumbersThread')
>>> t.start()
displayNumbersThread
0
1
```

```
2
3
4
5
>>> t.join()
>>> print("displayNumbers function finished")
displayNumbers function finished
```

Code 121: Simple threading via a function

As demonstrated above, the thread object `t` is dispatched via the `.start()` method, which schedules the thread for execution and returns immediately, allowing the main thread to continue concurrently. It is important to distinguish `.start()` from the lower-level `.run()` method: whereas `.start()` delegates execution to a newly spawned OS-level thread, `.run()` invokes the target callable directly on the *calling* thread, producing strictly **sequential rather than concurrent behaviour**.

The `.join()` method causes the main thread to halt until `t` has completed execution, thereby synchronising the two threads before control advances to subsequent statements. In this example, the main thread is suspended until `displayNumbers` returns, after which the final print statement is reached.

Finally, observe that invoking `threading.current_thread().name` from within `t`'s execution context yields `displayNumbersThread`, reflecting the identifier supplied via the `name` parameter at construction time. This illustrates that each `Thread` instance maintains its own execution context, including thread-local identity metadata accessible at runtime.

Core Concept 11.4: Threading Through Class Methods

Analogously to the function-based approach demonstrated in Core Concept 11.3, threads may equally be dispatched by targeting instance methods of a class.

```
>>> import threading
>>> from time import sleep # the sleep function simply makes the program wait

>>> class MyThread:
...     def displayNumbers(self):
...         i = 0
...         print(threading.current_thread().name)
...         while i <= 3:
...             print(i)
...             i+=1
...
>>> obj = MyThread()

>>> t1 = threading.Thread(target=obj.displayNumbers,name='displayNumbers1')
>>> t1.start()
displayNumbers1
0
1
2
3
>>> sleep(0.5)

>>> t2 = threading.Thread(target=obj.displayNumbers,name='displayNumbers2')
>>> t2.start()
displayNumbers2
0
1
```

```
2
3
>>> t1.join()
>>> t2.join()
```

Code 122: Multi-threading via a class method

The `sleep(0.5)` calls serve to introduce a deliberate delay between the dispatch of `t1` and `t2`, imposing an approximate execution ordering. In their absence, the relative scheduling of the two threads is delegated entirely to the Python thread scheduler, which provides no ordering guarantees—the threads may interweave arbitrarily or execute in an effectively simultaneous fashion depending on the underlying system load.

Core Concept 11.5: Common Producer/Consumer Pattern

The producer-consumer pattern is one of the most canonical demonstrations of multi-threaded coordination, wherein two threads operate concurrently on a shared resource.

```
>>> import threading
>>> import time

>>> class Producer:
...     def __init__(self):
...         self.products = []
...         self.ordersplaced = False
...     def produce(self):
...         for i in range(1,5):
...             self.products.append('Product'+str(i))
...             time.sleep(1)
...             print('Item Added')
...         self.ordersplaced = True
...
>>> class Consumer:
...     def __init__(self,prod):
...         self.prod = prod
...     def consume(self):
...         while self.prod.ordersplaced == False:
...             print("Waiting for the orders")
...             time.sleep(0.4)
...         print('Orders Shipped {}'.format(self.prod.products))
...
>>> p = Producer()
>>> c = Consumer(p)

>>> t1 = threading.Thread(target = p.produce)
>>> t2 = threading.Thread(target = c.consume)

>>> t1.start()
>>> t2.start()
Waiting for the orders
>>> t1.join()
Waiting for the orders
Waiting for the orders
Item Added
Waiting for the orders
Waiting for the orders
Item Added
Waiting for the orders
Waiting for the orders
Waiting for the orders
Item Added
```

```
Waiting for the orders
Waiting for the orders
Item Added
>>> t2.join()
Orders Shipped ['Product1', 'Product2', 'Product3', 'Product4']
```

Code 123: Multi-threading consumer & producer pattern

As illustrated in Code 123, the consumer thread `t2` continuously polls the shared `ordersplaced` flag within a busy-wait loop, deferring execution of the shipment statement until the flag evaluates to `True`. This state transition is driven by the producer thread `t1`, which iteratively populates the product list via its `for` loop before setting `ordersplaced = True` upon completion.

11.2 Locks

Core Concept 11.6: Locks

When multiple threads access a shared resource concurrently, conditions may arise in which overlapping operations produce inconsistent or corrupted state. To prevent this, Python's `threading` module provides the `Lock()` class, which enforces **mutual exclusion** by ensuring that at most one thread may execute a designated critical section at any given time.

```
>>> import threading

>>> class BookMyBus:
...     def __init__(self,availableSeats):
...         self.availableSeats = availableSeats
...         self.l = threading.Lock()
...     def buy(self,seatsRequested):
...         self.l.acquire() #starting the locked environment
...         print('Total seats available: {}'.format(self.availableSeats))
...         if(self.availableSeats>=seatsRequested):
...             print('Confirming a seat')
...             print('Processing the payment')
...             print('Printing the Ticket')
...             self.availableSeats-=seatsRequested
...         else:
...             print('Sorry. No seats available')
...         self.l.release() #ending the locked environment
...
>>> obj = BookMyBus(10)

>>> t1 = threading.Thread(target=obj.buy,args=(3,))
>>> t2 = threading.Thread(target=obj.buy,args=(4,))
>>> t3 = threading.Thread(target=obj.buy,args=(4,))

>>> t1.start()
Total seats available: 10
Confirming a seat
Processing the payment
Printing the Ticket
>>> t2.start()
Total seats available: 7
Confirming a seat
Processing the payment
Printing the Ticket
>>> t3.start()
Total seats available: 3
```

```
Sorry. No seats available
>>> t1.join()
>>> t2.join()
>>> t3.join()
```

Code 124: Threading locks enforcing mutual exclusion

The critical section—the entirity of the `buy` method—is surrounded by paired calls to `.acquire()` and `.release()`. When a thread invokes `.acquire()`, it attempts to take ownership of the lock: if the lock is currently unheld, the thread acquires it and proceeds into the critical section; if another thread already holds the lock, the calling thread blocks until the lock is relinquished. Once the protected operations are complete, `.release()` surrenders ownership of the lock, allowing the next waiting thread to acquire it and enter the critical section in turn.

Core Concept 11.7: Thread Synchronisation

Thread synchronisation (*communication*) is facilitated through the `Condition` class and its associated methods, `wait()` and `notify()`; these methods must be invoked within a locked context in order to prevent data corruption.

```
>>> import threading
>>> import time

>>> class Producer:
...     def __init__(self):
...         self.products = []
...         self.c = threading.Condition() # the Condition class already has the lock
 ↪  method in-built into it, thus you don't need to invoke a separate lock method
...     def produce(self):
...         self.c.acquire() # locked environment setting through the Condition class
...         for i in range(1,5):
...             self.products.append('Product'+str(i))
...             time.sleep(1)
...             print('Item Added: Product' + str(i))
...         self.c.notify() # where the program ends and notifies the consumer
...         self.c.release()
...
>>> class Consumer:
...     def __init__(self,prod):
...         self.prod = prod
...     def consume(self):
...         self.prod.c.acquire()
...         self.prod.c.wait()
...         self.prod.c.release()
...         print('Orders Shipped: {}'.format(self.prod.products))
... p = Producer()
  File "<stdin>", line 9
    p = Producer()
    ^
SyntaxError: invalid syntax
>>> c = Consumer(p)

>>> t1 = threading.Thread(target = p.produce)
>>> t2 = threading.Thread(target = c.consume)

>>> t1.start()
>>> t2.start()
Orders Shipped ['Product1', 'Product2', 'Product3', 'Product4', 'Product1']
```

```
38
39 >>> t1.join()
40 Item Added
41 Item Added
42 Item Added
43 Item Added
44 >>> t2.join()
```

Code 125: Threading using the `Condition` class

Consider the scenario in which the Python runtime schedules the `t2` thread (*consumer thread*) first; upon entering `consume()`, it acquires the lock and immediately finds the product list empty, as the producer has not yet executed. This results in a deadlock: the lock remains held within the `consume()` block, preventing the `produce()` function from acquiring it, since both threads contend over the same underlying `Condition` instance `self.c` and `self.prod.c`.

To prevent the above deadlock scenario, the `wait()` method—as seen in Code 125—performs three critical operations.

(1) It **automically releases the lock** held within the `acquire()` / `release()` block of the `consume()` function, returning it to the general pool so that the producer thread may acquire it.

(2) It suspends the calling thread and places it into an internal waiting queue managed by the `Condition` object.

(3) It remains suspended in the waiting queue until explicitly signalled by a call to `notify()`.

Once the lock is released back to the pool, the scheduler grants it to the `t1` thread (*producer thread*), which enters `produce()`, populates the product list, and completes its work. Prior to calling `release()`, the producer invokes `notify()`, which promotes the consumer thread `t2` from the waiting queue back to a runnable state, subsequently allowing it to acquire the lock and correctly print the shipped orders.

In summary, the `wait()`–`notify()` mechanism is a tool designed explicitly to handle unpredictable scheduling order arising from **race conditions** (*the Free-For-All race*), and to enforce a correct, deterministic execution sequence across cooperating threads.

11.3 Queuing

Core Concept 11.8: Queuing

The `queue` module provides a thread-safe mechanism for passing objects between threads, offering a simpler and more robust alternative implementation of the producer-consumer pattern with locks (*Core Concept 11.6*).

```
1  >>> import random
2  >>> import time
3  >>> import queue
4  >>> import threading
5
6  >>> def producer(q):
7  ...     for _ in range(2):
8  ...         print('Producing')
9  ...         q.put(random.randint(1,50)) #putting the random integer into a queue
10 ...         print('Produced')
11 ...         time.sleep(0.5)
12 ...
```

```
>>> def consumer(q):
...     for _ in range(2):
...         print('Ready to consume')
...         print('Consume Data: {}'.format(q.get())) #returning the queued integer
...         time.sleep(0.5)
...
>>> q = queue.Queue()
>>> t1 = threading.Thread(target=consumer,args=(q,))
>>> t2 = threading.Thread(target=producer,args=(q,))

>>> t1.start()
Ready to consume
>>> t2.start()
Producing
Produced
>>> t1.join()
Consume Data: 14
Producing
Produced
Ready to consume
Consume Data: 23
>>> t2.join()
```

Code 126: Queueing using `Queue()`

As illustrated in Code 126, the `Queue()` class internally manages all requisite locking, ensuring that concurrent `put()` and `get()` operations across multiple threads remain free from race conditions and data corruption. Specifically, the `put()` method enqueues the random integer generated by `producer()` into the shared `q` object, while the `get()` method dequeues and returns the first item in FIFO order for consumption by `consumer()`, which outputs the retrieved value via a `print` statement.

Core Concept 11.9: Alternative Queue Ordering

Beyond the default FIFO ordering, the `queue` module exposes alternative queue classes that govern the order in which elements are dequeued, providing developers with flexible retrieval strategies suited to different scheduling requirements.

```
>>> import queue

>>> lq = queue.LifoQueue() # last in first out
>>> lq.put(400)
>>> lq.put(100)
>>> lq.put(500)
>>> lq.put(50)
>>> while not lq.empty(): # this is another function to specify an empty queue
...     print(lq.get(), end=' ')
... print()
  File "<stdin>", line 3
    print()
    ^^^^^
SyntaxError: invalid syntax

>>> pq = queue.PriorityQueue() #in order
>>> pq.put(400)
>>> pq.put(100)
>>> pq.put(500)
>>> pq.put(50)
```

```
>>> while not pq.empty():
...     print(pq.get(), end=' ')
... print()
  File "<stdin>", line 3
    print()
    ^^^^^
SyntaxError: invalid syntax

>>> pq = queue.PriorityQueue()
>>> pq.put((400,'John')) # if the item in the queue is a str, then make a tuple with a int in the first index and it will use that number to define the order
>>> pq.put((100,'Bob'))
>>> pq.put((500,'Ahmed'))
>>> pq.put((50,'Bharath'))
>>> while not pq.empty():
...     print(pq.get()[1], end=' ') #insert the index to choose only the name in the prioritized order
...
Bharath Bob John Ahmed
```

Code 127: Custom queue ordering

As demonstrated in Code 127, the `LifoQueue` class implements a last-in, first-out retrieval policy, whereby the most recently enqueued element is always dequeued first. The `PriorityQueue` class, by contrast, dequeues elements in ascending order of their priority value; when the enqueued items are non-numeric, a tuple of the form `(priority, item)` may be supplied, whereupon the queue uses the integer at index zero to determine the retrieval order, with only the associated item at index one being extracted for use.

11.4 Daemon Threads

Core Concept 11.10: Daemon Threads

A **daemon thread** is a thread that runs silently in the background and is automatically terminated by the Python runtime as soon as all non-daemon threads have completed execution. Unlike standard threads, daemon threads are not awaited by the interpreter upon program exit, meaning any work they have not yet completed is abandoned without notice when the main thread finishes.

```
>>> import threading
>>> import time

>>> def background_task():
...     while True:
...         print('Daemon running...')
...         time.sleep(0.5)
...
>>> def main_task():
...     for i in range(3):
...         print('Main thread working: step {}'.format(i))
...         time.sleep(1)
...
>>> t1 = threading.Thread(target=background_task)
>>> t1.daemon = True # designates t1 as a daemon thread
>>> t2 = threading.Thread(target=main_task)

>>> t1.start()
Daemon running...
>>> t2.start()
Main thread working: step 0
```

```
>>> t2.join() # only the non-daemon thread is joined; once t2 finishes, t1 is killed automatically
Daemon running...
Main thread working: step 1
Daemon running...
Daemon running...
Main thread working: step 2
Daemon running...
Daemon running...
```

Code 128: Daemon thread termination behaviour

As illustrated in Code 128, the daemon flag is set via the `.daemon` attribute prior to invoking `start()`; assigning it thereafter raises a `RuntimeError`. Once the non-daemon thread `t2` completes its iteration and `join()` returns, the Python runtime terminates the process entirely, forcibly halting the daemon thread `t1` regardless of its current state within the `while` loop.

It is worth noting that daemon threads are deliberately not joined, as doing so would block the main thread indefinitely given their non-terminating nature. This distinguishes them from standard threads, where `join()` is used to ensure orderly completion.

12 Third-Party Libraries

12.1 Package Management

pip and PyPI conda

12.2 Third-Party Libraries

packages requests https://requests.readthedocs.io/en/latest/ seaborn https://seaborn.pydata.org/generated/seaborn.objects.Plot.ht
IPython.display https://ipython.readthedocs.io/en/stable/api/generated/IPython.display.html scikit https://scikit-learn.org/1.5/api/index.html statsmodels https://www.statsmodels.org/stable/api.html scipy https://docs.scipy.org/doc/scipy/refe

12.2.1 Data Science

numpy https://numpy.org/doc/stable/reference/ pandas https://pandas.pydata.org/docs/reference/index.html matplotlib https://matplotlib.org/stable/api/index.html

12.2.2 placeholder

13 Other

del input() print sep end